

Spring And Spring Boot

What is Spring and Spring Boot Questions and when too use what

What is Spring?

Spring Framework is a powerful Java framework used to build enterprise applications. It provides tools and features for:

- Dependency Injection (DI)
- Inversion of Control (IoC)
- Web Applications
- Database connectivity
- Security
- REST APIs
- Microservices

Spring reduces boilerplate code and helps build loosely coupled, maintainable applications.

Core Concepts of Spring

1. IOC (Inversion of Control)

Instead of creating objects manually using `new`, Spring creates and manages objects for you.

Example:

```
Java
Student s = new Student(); // Manual object creation
```

Spring manages this automatically using the IoC container.

2. Dependency Injection (DI)

Injecting required objects into another object.

Example:

```
Java
class Car {
    Engine engine;

    Car(Engine engine){
```

```

        this.engine = engine;
    }
}

```

Spring injects **Engine** automatically.

Types:

- Constructor Injection
- Setter Injection
- Field Injection

Constructor Injection is preferred in interviews.

3. Bean

A Java object managed by Spring container.

```

Java
@Component
public class Employee {
}

```

4. Spring Container

Responsible for:

- Creating objects
- Managing lifecycle
- Injecting dependencies

Types:

- BeanFactory
- ApplicationContext

Most used:

```

Java
ApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");

```

Important Spring Modules

Module	Purpose
Spring Core	IoC, DI
Spring MVC	Web applications

Module	Purpose
Spring JDBC	Database operations
Spring ORM	Hibernate integration
Spring AOP	Aspect Oriented Programming
Spring Security	Authentication & Authorization
Spring Data JPA	Database abstraction
Spring Batch	Batch processing

What is Spring Boot?

Spring Boot is built on top of Spring.

It simplifies Spring application development by:

- Removing complex configuration
- Providing auto-configuration
- Providing embedded servers
- Faster setup for production-ready apps

Why Spring Boot Was Introduced

Traditional Spring required:

- XML configuration
- Dispatcher servlet setup
- Server deployment
- Dependency management

Spring Boot solves this.

Spring vs Spring Boot

Feature	Spring	Spring Boot
Configuration	Manual	Auto
Server	External Tomcat	Embedded Tomcat
Setup Time	More	Very Fast
XML Usage	More	Minimal
Dependency Management	Manual	Starter dependencies
Production Ready	Extra setup	Built-in
Microservices	Difficult	Easy

Example

Spring Application

Requires:

- web.xml
- dispatcher servlet
- XML config
- server setup

Large configuration.

Spring Boot Application

```
Java
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Run directly.

Important Spring Boot Features

1. Auto Configuration

Automatically configures application based on dependencies.

Example:

If MySQL dependency exists → datasource auto-configured.

2. Starter Dependencies

Example:

```
XML
spring-boot-starter-web
```

Contains:

- Spring MVC
- Jackson
- Embedded Tomcat

3. Embedded Server

No need to deploy WAR file separately.

Supports:

- Tomcat
- Jetty
- Undertow

4. Actuator

Production monitoring endpoints.

Example:

```
Bash
/actuator/health
```

5. Spring Boot DevTools

Automatic restart during development.

Spring Boot Architecture Flow

Client → Controller → Service → Repository → Database

Common Annotations

Annotation	Purpose
@Component	Generic Bean
@Service	Service Layer
@Repository	DAO Layer
@Controller	MVC Controller
@RestController	REST API

Annotation	Purpose
@Autowired	Dependency Injection
@Configuration	Configuration Class
@Bean	Manual Bean Creation
@SpringBootApplication	Main Boot Annotation

Important Layers in Spring Boot

Controller Layer

Handles requests.

```
Java
@RestController
public class UserController {
}
```

Service Layer

Business logic.

```
Java
@Service
public class UserService {
}
```

Repository Layer

Database operations.

```
Java
@Repository
public interface UserRepo extends JpaRepository<User, Long> {
}
```

Spring Boot Project Flow

1. Request comes
2. Controller receives
3. Calls Service
4. Service calls Repository
5. Repository interacts with DB

6. Response returned

Spring MVC Flow

Browser Request



DispatcherServlet



Controller



Service



DAO



View/JSON Response

What is DispatcherServlet?

Front controller of Spring MVC.

It handles all incoming requests.

What is AOP?

Aspect Oriented Programming.

Used for:

- Logging
- Security
- Transactions
- Exception handling

Concepts:

- Aspect
 - Advice
 - Join Point
 - Pointcut
-

What is Spring Bean Lifecycle?

1. Bean creation

2. Dependency injection
 3. init()
 4. Bean used
 5. destroy()
-

What is Spring Security?

Spring Security provides:

- Authentication
 - Authorization
 - JWT
 - OAuth2
 - CSRF protection
-

What is Spring Data JPA?

Spring Data JPA simplifies database operations.

Instead of writing SQL:

```
Java  
findByEmail(String email)
```

Spring generates query automatically.

What is Hibernate?

Hibernate is an ORM framework.

Converts:

- Java Objects ↔ Database Tables
-

Spring Boot Interview Questions

Basic Level

1. Difference between Spring and Spring Boot?

- Spring → Framework
 - Spring Boot → Simplified version with auto configuration
-

2. What is Dependency Injection?

Injecting object dependencies rather than creating manually.

3. What are Beans?

Objects managed by Spring container.

4. What is IOC?

Control transferred from programmer to Spring container.

5. Difference between @Component, @Service, @Repository?

All are stereotype annotations.

- **@Component** → Generic
- **@Service** → Business layer
- **@Repository** → DAO layer

6. What is @Autowired?

Automatically injects dependency.

7. Constructor Injection vs Setter Injection?

Constructor	Setter
Mandatory dependency	Optional dependency
Immutable	Mutable
Preferred	Less preferred

8. What is @SpringBootApplication?

Combination of:

```
Java
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

9. What is embedded Tomcat?

Tomcat packaged inside application.

10. What is Actuator?

Production monitoring tool.

Intermediate Questions

11. Explain Bean Scope

Scope	Meaning
singleton	One object
prototype	New object every time
request	Per HTTP request
session	Per session

12. What is BeanFactory vs ApplicationContext?

BeanFactory	ApplicationContext
Basic container	Advanced container
Lazy loading	Eager loading

13. What is Spring MVC?

Web framework based on MVC pattern.

14. Difference between @Controller and @RestController?

@Controller	@RestController
Returns view	Returns JSON
MVC apps	REST APIs

15. What is JPA?

Java Persistence API specification.

Hibernate is implementation of JPA.

16. What is ORM?

Object Relational Mapping.

17. What is Hibernate Session?

Connection between Java app and database.

18. Difference between GET and POST?

GET	POST
Fetch data	Send data
Visible params	Hidden body
Idempotent	Non-idempotent

Advanced Interview Questions

19. What happens internally when Spring Boot starts?

1. Main method executes
2. SpringApplication.run()
3. IOC container created
4. Beans initialized
5. Embedded server starts
6. Application ready

20. How does Auto Configuration work?

Uses:

```
Java
@EnableAutoConfiguration
```

Reads dependencies and configures automatically.

21. What is Lazy Initialization?

Bean created only when needed.

```
Java
@Lazy
```

22. What is Circular Dependency?

A depends on B

B depends on A

Causes bean creation issue.

23. Explain Exception Handling in Spring Boot

Using:

```
Java
@ControllerAdvice
@ExceptionHandler
```

24. What is Microservice Architecture?

Application split into smaller independent services.

Spring Boot is widely used for microservices.

25. What is JWT?

JSON Web Token used for stateless authentication.

When to Use Spring vs Spring Boot

Use Spring Framework When:

- You need very fine-grained configuration
- Legacy enterprise applications
- Large monolithic applications
- Existing Spring MVC projects

Use Spring Boot When:

- Building REST APIs
- Microservices
- Modern enterprise apps
- Fast development
- Cloud-native applications

Most companies today prefer Spring Boot.

Real Industry Stack

Typical backend stack:

- Java
 - Spring Boot
 - Hibernate/JPA
 - MySQL/PostgreSQL
 - Redis
 - Kafka
 - Docker
 - Kubernetes
-

Most Asked Topics for 2–4 Years Experience

Focus heavily on:

- Dependency Injection
 - Bean lifecycle
 - Spring Boot annotations
 - REST APIs
 - JPA/Hibernate
 - Transactions
 - Exception handling
 - Security/JWT
 - Microservices basics
 - Collections + Streams + Multithreading
-

Best Order to Learn

1. Core Java
2. Collections
3. Streams
4. JDBC
5. Servlets
6. Spring Core
7. Spring MVC
8. Spring Boot
9. JPA/Hibernate
10. REST APIs
11. Security/JWT
12. Microservices

Since you're already working as a Java developer at Aurionpro, focus more on:

- Spring Boot internals
- Transactions
- Hibernate optimization
- API security
- Debugging production issues
- Microservices communication

These are asked heavily in real interviews.

what is Ioc and Dependency Injection examples and Interview Questions asked

What is IoC (Inversion of Control)?

Spring Framework IoC means:

The control of creating and managing objects is transferred from the programmer to the Spring container.

Normally in Java:

```
Java
Engine engine = new Engine();
Car car = new Car(engine);
```

You are manually creating objects.

With Spring IoC:

- Spring creates objects
- Spring manages lifecycle
- Spring injects dependencies automatically

So control is inverted from developer → Spring container.

Real Life Example of IoC

Without IoC

You cook food yourself:

- Buy ingredients
- Prepare everything
- Manage all steps

You control everything.

With IoC

You order from restaurant:

- Restaurant prepares food
- You only consume it

Control shifted to restaurant.

Same in Spring:

- Spring creates/manages objects
 - You only use them
-

What is Dependency?

If one class needs another class to work → dependency.

Example:

```
Java
class Car {
    Engine engine;
}
```

Car depends on Engine.

What is Dependency Injection (DI)?

Dependency Injection means:

Supplying dependent objects from outside instead of creating them inside class.

Without Dependency Injection

```
Java
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class Car {
    Engine engine = new Engine(); // tightly coupled

    void drive() {
        engine.start();
        System.out.println("Car driving");
    }
}
```

Problem:

- Car itself creates Engine
- Tight coupling
- Hard to test
- Hard to replace engine

With Dependency Injection

```
Java
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private Engine engine;

    Car(Engine engine) {
        this.engine = engine;
    }

    void drive() {
        engine.start();
    }
}
```


Now:

- Engine comes from outside
- Loose coupling
- Easy testing
- Easy replacement

This is Dependency Injection.

How Spring Performs DI

Spring container:

1. Creates Engine object
2. Creates Car object
3. Injects Engine into Car

Automatically.

Types of Dependency Injection

1. Constructor Injection (Most Preferred)

```
Java
@Component
class Engine {
}

@Component
class Car {
    private Engine engine;

    @Autowired
    Car(Engine engine) {
        this.engine = engine;
    }
}
```

Best because:

- Mandatory dependency
 - Immutable
 - Easier testing
-

2. Setter Injection

```
Java
```

```
@Component
class Car {

    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}
```

Used for optional dependencies.

3. Field Injection

```
Java
@Component
class Car {

    @Autowired
    private Engine engine;
}
```

Easy but NOT preferred in interviews.

Problems:

- Hard to test
- Reflection based
- Less clean

IoC Container in Spring

Spring container manages beans.

Main containers:

- BeanFactory
- ApplicationContext

Most used:

```
Java
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

What is a Bean?

A bean is:

Object managed by Spring container.

Example:

```
Java
@Component
class Student {
}
```

Spring creates Student bean.

How Beans Are Created

Using:

- @Component
- @Service
- @Repository
- @Controller
- @Bean

Example Flow of IoC + DI

```
Java
@Component
class Engine {
}

@Component
class Car {
    private Engine engine;

    Car(Engine engine) {
        this.engine = engine;
    }
}
```

What happens internally:

1. Spring scans classes
2. Creates Engine bean
3. Creates Car bean
4. Injects Engine into Car

Tight Coupling vs Loose Coupling

Tight Coupling

```
Java
Engine engine = new Engine();
```

Bad because:

- Hardcoded dependency
 - Hard to modify
-

Loose Coupling

```
Java
Car(Engine engine)
```

Good because:

- Flexible
 - Replaceable
 - Testable
-

Why IoC and DI Are Important

Advantages:

- Loose coupling
 - Better testing
 - Easier maintenance
 - Reusability
 - Scalability
 - Cleaner code
-

Real Interview Example

Without DI

```
Java
class NotificationService {
    EmailService email = new EmailService();
}
```

Cannot easily switch to SMS.

With DI

```
Java
class NotificationService {
    private MessageService service;

    NotificationService(MessageService service) {
        this.service = service;
    }
}
```

Now:

- EmailService
- SMSService
- WhatsAppService

All can be injected easily.

Important IoC Interview Questions

1. What is IoC?

IoC means transferring control of object creation and management from programmer to Spring container.

2. What is Dependency Injection?

Injecting required dependencies from outside instead of creating internally.

3. Difference between IoC and DI?

IoC	DI
Principle	Design pattern
Control inversion	Way to achieve IoC
Broad concept	Specific implementation

Simple answer:

- IoC = concept
- DI = implementation of IoC

4. What are benefits of DI?

- Loose coupling
 - Easier testing
 - Better maintainability
 - Flexible architecture
-

5. Types of Dependency Injection?

- Constructor Injection
 - Setter Injection
 - Field Injection
-

6. Which Injection is Preferred?

Constructor Injection.

Because:

- Mandatory dependencies
 - Immutable objects
 - Easier unit testing
-

7. Why is Field Injection not preferred?

Problems:

- Hard to test
 - Uses reflection
 - Cannot create immutable objects
-

8. What is a Spring Bean?

Object managed by Spring IoC container.

9. What is Bean Scope?

Scope	Meaning
singleton	One object

Scope	Meaning
prototype	Multiple objects
request	Per HTTP request
session	Per user session

10. What is ApplicationContext?

Advanced Spring IoC container.

Provides:

- Bean creation
- Dependency injection
- Events
- Internationalization

11. BeanFactory vs ApplicationContext?

BeanFactory	ApplicationContext
Basic container	Advanced container
Lazy loading	Eager loading
Less features	More features

12. What happens internally when Spring starts?

1. Container starts
2. Scans classes
3. Creates beans
4. Resolves dependencies
5. Injects dependencies
6. Application ready

13. What is Autowiring?

Automatic dependency injection by Spring.

Using:

```
Java
@Autowired
```

14. What is Circular Dependency?

Example:

```
Java
A depends on B
B depends on A
```

Causes bean creation problem.

15. What is Qualifier?

Used when multiple beans of same type exist.

```
Java
@Autowired
@Qualifier("emailService")
private MessageService service;
```

Frequently Asked Scenario Questions

Scenario 1

Why Constructor Injection over Field Injection?

Answer:

- Better immutability
 - Easier testing
 - Clear mandatory dependencies
 - Avoid reflection
-

Scenario 2

If multiple beans exist, how does Spring decide?

Uses:

- @Primary

- @Qualifier

Scenario 3

What if dependency is not found?

Exception:

```
Java
NoSuchBeanDefinitionException
```

Scenario 4

Can Spring inject interface references?

Yes.

Example:

```
Java
interface PaymentService {
}

@Service
class UpiPaymentService implements PaymentService {
}
```

Spring injects implementation automatically.

Best Way to Explain in Interview

Short crisp answer:

IoC means Spring controls object creation and lifecycle instead of developer manually creating objects.

Dependency Injection is a technique where Spring injects dependent objects automatically, helping achieve loose coupling.

Most Asked Practical Questions for Experienced Developers

Since you're working with Java enterprise applications at Aurionpro, expect these:

- How does @Autowired work internally?
- What happens if two beans of same type exist?
- Why constructor injection preferred?

- Bean lifecycle?
- Singleton bean thread safety?
- Circular dependency handling?
- How does Spring create proxies?
- Difference between `@Component` and `@Bean`?
- Can Spring inject static fields?
- Lazy vs eager initialization?
- Prototype bean inside singleton?

These are very common in 2–5 year Java/Spring interviews.

what is Reflection in java with example

What is Reflection in Java?

Java Reflection is a feature in Java that allows a program to:

- Inspect classes at runtime
- Access methods
- Access fields
- Access constructors
- Create objects dynamically
- Invoke methods dynamically

Even if details are not known during compile time.

Simple Definition

Reflection means examining and manipulating classes, methods, fields, and objects at runtime.

Why Reflection is Used

Used heavily in frameworks like:

- Spring Framework
- Hibernate
- JUnit

These frameworks dynamically:

- Create objects
- Inject dependencies
- Call methods
- Read annotations

without manually writing object creation code.

Real Life Example

Imagine:

- You don't know what TV model user bought.
- But you still want to:
 - Turn it on
 - Change volume
 - Access settings

Reflection lets Java inspect object details dynamically.

Basic Reflection Classes

Main package:

```
Java
java.lang.reflect
```

Important classes:

- **Class**
- **Method**
- **Field**
- **Constructor**

Example 1 — Get Class Information

```
Java
class Student {
}

public class Main {
    public static void main(String[] args) {
        Class c = Student.class;
        System.out.println(c.getName());
    }
}
```

```
}  
}
```

Output:

```
Java  
Student
```

Example 2 — Get Methods Dynamically

```
Java  
import java.lang.reflect.Method;  
  
class Student {  
    public void display() {  
        System.out.println("Hello");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Class c = Student.class;  
        Method[] methods = c.getDeclaredMethods();  
        for(Method m : methods) {  
            System.out.println(m.getName());  
        }  
    }  
}
```

Output:

```
Java  
display
```

Example 3 — Invoke Method Dynamically

```
Java  
import java.lang.reflect.Method;  
  
class Student {  
    public void show() {  
        System.out.println("Method called");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Student s = new Student();  
        Class c = s.getClass();  
        Method m = c.getMethod("show");  
        m.invoke(s);  
    }  
}
```

```
}  
}
```

Output:

```
Java  
Method called
```

Example 4 — Access Private Field

```
Java  
import java.lang.reflect.Field;  
  
class Student {  
    private String name = "Lawrence";  
}  
  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Student s = new Student();  
        Class c = s.getClass();  
        Field f = c.getDeclaredField("name");  
        f.setAccessible(true);  
        System.out.println(f.get(s));  
    }  
}
```

Output:

```
Java  
Lawrence
```

Example 5 — Create Object Dynamically

```
Java  
class Employee {  
    Employee() {  
        System.out.println("Object created");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Class c = Employee.class;  
        Object obj = c.getDeclaredConstructor().newInstance();  
    }  
}
```

Output:

Java
Object created

How Spring Uses Reflection

In Spring Framework:

When you write:

```
Java
@Autowired
private UserService userService;
```

Spring:

1. Scans class
2. Detects `@Autowired`
3. Uses reflection
4. Injects object into field

Even private fields can be injected because reflection bypasses access modifiers.

How Hibernate Uses Reflection

Hibernate uses reflection to:

- Read entity fields
- Map objects to tables
- Access private variables

Example:

```
Java
@Entity
class User {
    private String name;
}
```

Hibernate accesses `name` using reflection.

Advantages of Reflection

Advantage	Description
Dynamic behavior	Runtime inspection
Framework development	Spring/Hibernate use it
Flexible code	Generic libraries

Advantage	Description
Access private members	Possible using reflection

Disadvantages of Reflection

Problem	Why
Slower	Runtime overhead
Security risk	Can access private data
Complex debugging	Hard to trace
Breaks encapsulation	Access modifiers bypassed

Important Methods in Reflection

Method	Purpose
<code>getClass()</code>	Get class object
<code>getMethods()</code>	Public methods
<code>getDeclaredMethods()</code>	All methods
<code>getFields()</code>	Public fields
<code>getDeclaredFields()</code>	All fields
<code>getConstructor()</code>	Constructor
<code>invoke()</code>	Call method
<code>newInstance()</code>	Create object

Difference Between Normal vs Reflection

Normal Method Call

```
Java
student.show();
```

Compile-time known.

Reflection Method Call

```
Java
```

```
Method m = c.getMethod("show");
m.invoke(student);
```

Runtime dynamic.

Real Interview Questions

1. What is Reflection in Java?

Reflection allows inspection and manipulation of classes, methods, constructors, and fields at runtime.

2. Why Reflection is Used?

Used in frameworks like:

- Spring
- Hibernate
- JUnit

for dynamic object handling.

3. Which package is used for Reflection?

```
Java
java.lang.reflect
```

4. Can Reflection access private members?

Yes.

Using:

```
Java
setAccessible(true)
```

5. Why Reflection is slower?

Because:

- Operations happen at runtime
- JVM optimizations are limited

6. Difference between `getMethods()` and `getDeclaredMethods()`?

Method	Returns
<code>getMethods()</code>	Public methods including inherited
<code>getDeclaredMethods()</code>	All methods of class only

7. How does Spring use Reflection?

Spring uses reflection to:

- Create beans
- Inject dependencies
- Process annotations

8. What is `Class` class?

Represents metadata of class.

Example:

```
Java
Class c = Student.class;
```

9. Can we create objects using Reflection?

Yes.

```
Java
c.getDeclaredConstructor().newInstance();
```

10. What are drawbacks of Reflection?

- Slow performance
- Security issues
- Complex maintenance

Most Important Interview Point

Very commonly asked:

“How does Spring inject private fields?”

Answer:

Spring uses Java Reflection to access and inject dependencies into private fields using `setAccessible(true)` internally.

Reflection + Annotation Example

```
Java
@Retention(RetentionPolicy.RUNTIME)
@interface MyAnnotation {
}

@MyAnnotation
class Test {
}
```

Frameworks inspect annotations using reflection.

Advanced Interview Topics

For experienced developers, prepare:

- Reflection vs Dynamic Proxy
- CGLIB Proxy
- JDK Proxy
- Annotation processing
- Runtime class loading
- Spring bean creation internals
- How `@Autowired` works internally
- Reflection performance optimization

These are commonly asked in Spring Boot interviews for 2–5 years experience.

How does `@Autowired` work internally?
 What happens if two beans of same type exist?
 What is Bean and explain Bean lifecycle?
 Circular dependency handling?
 Difference between `@Component` and `@Bean`?
 Lazy vs eager initialization?

1. How does `@Autowired` work internally?

In Spring Framework, `@Autowired` is used for automatic dependency injection.

Example:

```
Java
@Service
class UserService {
}

@RestController
class UserController {

    @Autowired
    private UserService userService;
}
```

Spring automatically injects `UserService`.

Internal Working of @Autowired

Step-by-Step Flow

Step 1 — Component Scanning

Spring scans classes using:

```
Java
@ComponentScan
```

It detects:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

and creates beans.

Step 2 — Bean Creation

Spring IoC container creates objects (beans).

Example:

```
Java
UserService userService = new UserService();
```

internally by container.

Step 3 — Dependency Resolution

Spring checks:

```
Java
@Autowired
private UserService userService;
```

It searches container for matching bean type.

Step 4 — Reflection Used

Spring uses Java Reflection internally:

```
Java
field.setAccessible(true);
field.set(object, dependencyObject);
```

Even private fields can be injected.

Important Internal Classes

Spring internally uses:

- **BeanFactory**
- **ApplicationContext**
- **AutowiredAnnotationBeanPostProcessor**

Very commonly asked.

Types of Autowiring

By Type (Default)

Spring searches matching type.

```
Java
@Autowired
private PaymentService paymentService;
```

By Name

Used with:

```
Java
@Qualifier("upiPaymentService")
```

2. What Happens if Two Beans of Same Type Exist?

Example:

```
Java
interface PaymentService {
}

@Service
class UpiPaymentService implements PaymentService {
}

@Service
class CardPaymentService implements PaymentService {
}
```

Now:

```
Java
@Autowired
private PaymentService paymentService;
```

Spring becomes confused.

Exception Thrown

```
Java
NoUniqueBeanDefinitionException
```

Because multiple beans found.

Solutions

Solution 1 — @Qualifier

```
Java
@Autowired
@Qualifier("upiPaymentService")
private PaymentService paymentService;
```

Explicit bean selection.

Solution 2 — @Primary

```
Java
@Primary
@Service
class UpiPaymentService implements PaymentService {
}
```

Default bean selected automatically.

Interview Tip

Priority:

```
@Qualifier > @Primary
```

If both exist → @Qualifier wins.

3. What is Bean?

A Bean is:

An object managed by Spring IoC container.

Example:

```
Java
@Component
class Employee {
}
```

Spring creates and manages this object.

Ways to Create Beans

Using Stereotype Annotations

```
Java
@Component
@Service
@Repository
@Controller
```

Using @Bean

```
Java
@Configuration
class AppConfig {

    @Bean
```

```
public Engine engine() {  
    return new Engine();  
}
```

Bean Lifecycle

Very important interview topic.

Complete Bean Lifecycle

1. Container Starts

Spring application starts.

2. Bean Instantiation

Object created.

```
Java  
new Employee();
```

3. Dependency Injection

Dependencies injected using:

```
Java  
@Autowired
```

4. BeanNameAware Methods

Spring sets bean name.

5. BeanPostProcessor (Before Init)

Spring processes bean before initialization.

6. Initialization Methods

Methods called:

- `@PostConstruct`
- `afterPropertiesSet()`
- custom init method

Example:

```
Java
@PostConstruct
public void init() {
    System.out.println("Init method");
}
```

7. Bean Ready for Use

Application uses bean.

8. Destruction Phase

When application closes:

- `@PreDestroy`
- destroy method

Example:

```
Java
@PreDestroy
public void destroy() {
    System.out.println("Bean destroyed");
}
```

Bean Lifecycle Diagram

```

Container Start
  ↓
Bean Created
  ↓
Dependencies Injected
  ↓
@PostConstruct
  ↓
Bean Ready
  ↓
@PreDestroy
  ↓
Bean Removed
  
```

4. Circular Dependency Handling

Very commonly asked.

What is Circular Dependency?

Example:

```
Java
class A {
    B b;
}

class B {
    A a;
}
```

A depends on B

B depends on A

Cycle created.

Constructor Injection Circular Dependency

```
Java
@Service
class A {
    A(B b){}
}

@Service
class B {
    B(A a){}
}
```

Spring cannot create objects.

Exception

```
Java
BeanCurrentlyInCreationException
```

Why?

To create A → need B

To create B → need A

Infinite loop.

Solutions

Solution 1 — Use @Lazy

```
Java
```

```
@Service
class A {

    A(@Lazy B b){}

}
```

Spring injects proxy object first.

Solution 2 — Setter Injection

Setter injection allows partial object creation.

```
Java
@Autowired
public void setB(B b){
    this.b = b;
}
```

Solution 3 — Redesign Architecture

Best real-world solution.

Circular dependencies usually indicate bad design.

Important Interview Point

Spring can handle:

- Setter injection circular dependency

Spring cannot handle:

- Constructor injection circular dependency

5. Difference Between @Component and @Bean

Very important.

@Component

Used on class directly.

```
Java
@Component
class Engine {

}
```

Spring automatically scans and creates bean.

@Bean

Used inside configuration class.

```
Java
@Configuration
class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine();
    }
}
```

You manually control bean creation.

Difference Table

@Component	@Bean
Class-level annotation	Method-level annotation
Automatic scanning	Manual bean creation
Used for own classes	Used for third-party classes
Simpler	More control

When to Use Which?

Use @Component

When:

- You own source code
- Automatic scanning needed

Use @Bean

When:

- Third-party library class
- Custom initialization needed

Example:

```
Java
@Bean
public ObjectMapper mapper() {
```

```
return new ObjectMapper();
}
```

6. Lazy vs Eager Initialization

Eager Initialization (Default)

Beans created during application startup.

```
Java
@Component
class Employee {
}
```

Created immediately.

Advantages

- Faster runtime access
- Early error detection

Disadvantages

- Slower startup
- More memory usage

Lazy Initialization

Bean created only when first needed.

```
Java
@Lazy
@Component
class Employee {
}
```

Created only on usage.

Advantages

- Faster startup
- Reduced memory initially

Disadvantages

- First request slower
- Errors delayed until runtime

Example

```
Java
@Autowired
@Lazy
private PaymentService paymentService;
```

Bean created only when accessed.

Important Interview Questions

1. How does @Autowired work internally?

Answer:

Spring scans beans, resolves dependencies by type, and injects them using reflection through **AutowiredAnnotationBeanPostProcessor**.

2. What if multiple beans exist?

Answer:

Spring throws **NoUniqueBeanDefinitionException**. Use **@Qualifier** or **@Primary**.

3. What is Bean lifecycle?

Explain:

- Instantiation
- Dependency Injection
- Initialization
- Usage
- Destruction

4. How Spring handles circular dependency?

Answer:

- Handles setter injection circular dependency
- Cannot handle constructor injection circular dependency directly

5. Difference between @Component and @Bean?

Answer:

- **@Component** → auto scanned
- **@Bean** → manually declared in configuration

6. Lazy vs Eager?

Lazy	Eager
Created on demand	Created at startup
Faster startup	Faster access

Most Asked Advanced Follow-Up Questions

Since you're working in enterprise Java at Aurionpro, also prepare:

- What is BeanPostProcessor?
- How does Spring create proxies?
- Singleton bean thread safety?
- Difference between prototype and singleton?
- What is CGLIB proxy?
- What happens during **SpringApplication.run()**?
- How does **@Transactional** work internally?
- Difference between JDK dynamic proxy and CGLIB?
- How does Spring Boot auto configuration work?

These are heavily asked in Spring Boot interviews for 2-5 years experience.

What is Spring MVC?

Explain with interview point of view

What is the Difference between Spring MVC and Spring and Spring Boot

What is Spring MVC?

Spring MVC is a web framework inside the Spring Framework used to build:

- Web applications
- REST APIs
- Enterprise applications

MVC stands for:

```
M → Model
V → View
C → Controller
```

It follows the MVC design pattern.

Simple Definition for Interview

Spring MVC is a web framework built on top of Spring that follows the Model-View-Controller architecture for handling HTTP requests and responses.

MVC Architecture

1. Model

Contains:

- Business logic
- Data
- Database interaction

Example:

```
Java
class User {
    private String name;
}
```

2. View

UI layer shown to user.

Examples:

- JSP
- HTML
- Thymeleaf

3. Controller

Handles requests from browser.

Example:

```
Java
@Controller
public class UserController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }
}
```

Flow of Spring MVC

```
Client Request
↓
DispatcherServlet
↓
Controller
↓
Service Layer
↓
DAO/Repository
↓
Database
↓
View Resolver
↓
Response Returned
```

Most Important Component — DispatcherServlet

Very commonly asked.

What is DispatcherServlet?

Front Controller of Spring MVC.

It receives ALL incoming requests.

Internal Flow of DispatcherServlet

1. Request comes
2. DispatcherServlet receives request
3. Finds correct controller
4. Calls controller method
5. Gets response

6. Resolves view
7. Returns response

Example of Spring MVC

Controller Example

```
Java
@Controller
public class HomeController {

    @RequestMapping("/hello")
    public String hello(Model model) {

        model.addAttribute("msg", "Welcome");

        return "home";
    }
}
```

What Happens Internally?

User hits:

```
/hello
```

DispatcherServlet:

- finds `hello()` method
- executes it
- adds model data
- returns `home.jsp`

Important Spring MVC Annotations

Annotation	Purpose
<code>@Controller</code>	MVC Controller
<code>@RestController</code>	REST API controller
<code>@RequestMapping</code>	URL mapping
<code>@GetMapping</code>	GET request
<code>@PostMapping</code>	POST request
<code>@RequestParam</code>	Query param
<code>@PathVariable</code>	URL variable
<code>@ModelAttribute</code>	Bind form data
<code>@ResponseBody</code>	Return JSON

@Controller VS @RestController

@Controller

Returns view:

```
Java
return "home";
```

Used for:

- JSP
- Thymeleaf pages

@RestController

Returns JSON directly.

```
Java
@RestController
class UserController {

    @GetMapping("/user")
    public User getUser() {
        return new User();
    }
}
```

Used for:

- REST APIs
- Microservices

Spring MVC Architecture Components

Component	Role
DispatcherServlet	Front controller
Controller	Handles request
Service	Business logic
DAO/Repository	Database
ViewResolver	Finds view
Model	Data

What is ViewResolver?

Converts logical name:

```
Java
home
```

into:

```
/WEB-INF/views/home.jsp
```

What is Model?

Used to pass data from controller to view.

Example:

```
Java
model.addAttribute("name", "Lawrence");
```

Exception Handling in Spring MVC

Using:

```
Java
@ControllerAdvice
@ExceptionHandler
```

Global exception handling.

Spring MVC Interview Questions

Basic Questions

1. What is Spring MVC?

Spring MVC is a web framework based on MVC architecture used for developing web applications and REST APIs.

2. What does MVC stand for?

```
Model
View
```

3. What is DispatcherServlet?

Front controller that handles all incoming requests in Spring MVC.

Very important answer.

4. Difference between @Controller and @RestController?

@Controller	@RestController
Returns View	Returns JSON
Web apps	REST APIs

5. What is ViewResolver?

Maps logical view name to actual view page.

6. What is Model?

Used to send data from controller to view.

7. What is @RequestMapping?

Maps URL to controller method.

8. Difference between GET and POST?

GET	POST
Fetch data	Send data
Data in URL	Data in body

Intermediate Questions

9. What happens when request comes in Spring MVC?

Flow:

```
Client
→ DispatcherServlet
→ Controller
→ Service
→ Repository
→ DB
→ Response
```

10. What is Front Controller Pattern?

Single controller handles all requests.

DispatcherServlet follows this pattern.

11. How does Spring MVC support REST APIs?

Using:

```
Java
@RestController
@ResponseBody
```

12. What is @PathVariable?

Extracts value from URL.

Example:

```
Java
@GetMapping("/user/{id}")
```

13. What is @RequestParam?

Reads query parameter.

Example:

```
Java
?name=Lawrence
```

Advanced Questions

14. How does DispatcherServlet work internally?

1. Receives request
 2. HandlerMapping finds controller
 3. HandlerAdapter executes method
 4. ViewResolver resolves view
 5. Response returned
-

15. What is HandlerMapping?

Maps request URL to controller method.

16. What is HandlerAdapter?

Invokes controller method.

17. How does Spring MVC handle exceptions?

Using:

```
Java
@ControllerAdvice
```

Difference Between Spring, Spring MVC, and Spring Boot

Very important interview topic.

1. Spring Framework

Spring Framework is the complete framework.

Contains:

- IoC
- DI
- AOP
- JDBC
- MVC
- Security

- ORM

It is the parent framework.

2. Spring MVC

Spring MVC is a module inside Spring Framework.

Used specifically for:

- Web applications
- HTTP request handling
- REST APIs

3. Spring Boot

Spring Boot simplifies Spring application development.

Provides:

- Auto configuration
- Embedded server
- Starter dependencies
- Faster setup

Difference Table

Feature	Spring	Spring MVC	Spring Boot
Type	Complete framework	Web module	Framework extension
Purpose	Enterprise apps	Web apps/APIs	Simplified Spring
Configuration	Heavy	Heavy	Minimal
Server	External	External	Embedded
XML	More	More	Very less
Setup	Complex	Complex	Fast
REST APIs	Possible	Yes	Very easy
Microservices	Hard	Hard	Easy

Relationship Between Them

```

Spring Framework
  ↓
Spring MVC

```

↓
Spring Boot uses both

Real Industry Usage

Traditional Old Projects

Used:

- Spring + Spring MVC
 - XML configuration
 - WAR deployment
-

Modern Projects

Use:

- Spring Boot + REST APIs
- Microservices
- Embedded Tomcat

Most companies today prefer Spring Boot.

Interview-Friendly Answer

Difference in One Line

Spring

Complete enterprise Java framework.

Spring MVC

Web framework inside Spring for handling HTTP requests.

Spring Boot

Tool/framework that simplifies Spring development using auto configuration.

Most Asked Real Interview Questions

Since you work on enterprise Java projects at Aurionpro, focus heavily on:

- DispatcherServlet internals
- Request lifecycle
- REST API flow
- Exception handling
- Validation
- Interceptors vs Filters
- Spring Boot auto configuration
- Difference between Monolith and Microservices
- WAR vs JAR deployment
- REST best practices
- HTTP status codes

These are extremely common for 2–5 year Spring Boot interviews.

What are servelets, tomcat and anyother which i need to know some configurations and all
What is WAR file and JAR file

What are Servlets?

Java Servlet Servlet is a Java program that runs on a web server and handles:

- HTTP requests
- HTTP responses

Servlets are the foundation of Java web applications.

Before Spring MVC and Spring Boot, developers used Servlets directly.

Simple Flow of Servlet

```

Browser Request
  ↓
Servlet
  ↓
Business Logic
  ↓
Response Returned
  
```

Example Servlet

```

Java
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
                        HttpServletResponse res) {
  
```

```

    }
    System.out.println("Request received");
}

```

Important Servlet Methods

Method	Purpose
<code>doGet()</code>	Handles GET request
<code>doPost()</code>	Handles POST request
<code>init()</code>	Called once when servlet created
<code>destroy()</code>	Called before servlet destroyed
<code>service()</code>	Dispatches requests

Servlet Lifecycle

Very important interview topic.

```

Servlet Loaded
  ↓
init()
  ↓
service()
  ↓
doGet()/doPost()
  ↓
destroy()

```

What is Tomcat?

Apache Tomcat is:

A web server + servlet container used to run Java web applications.

Tomcat executes:

- Servlets
- JSP
- Spring MVC apps
- Spring Boot apps

Why Tomcat is Needed

Java code alone cannot handle browser requests.

Tomcat:

- Receives HTTP request

- Executes servlet
- Sends response

Real Flow

```

Browser
↓
Tomcat Server
↓
Servlet/Spring Controller
↓
Response

```

Difference Between Web Server and Servlet Container

Web Server	Servlet Container
Handles static content	Handles Java servlets
HTML/CSS/JS	Java execution

Tomcat does both.

Common Java Servers

Server	Usage
Apache Tomcat	Most common
Jetty	Lightweight
WildFly	Enterprise
JBoss EAP	Enterprise
WebLogic Server	Large enterprise
WebSphere Application Server	Banking/legacy

What is JSP?

JavaServer Pages JSP is used for dynamic HTML generation.

Example:

```

JSP
<h1>Hello <%= name %></h1>

```

Old Spring MVC projects heavily used JSP.

Now mostly replaced by:

- REST APIs

- Angular
- React

Important Servlet Configurations

1. web.xml

Old deployment descriptor.

Used for:

- Servlet mapping
- Filters
- Listener config

Example:

```
XML
<servlet>
  <servlet-name>hello</servlet-name>
  <servlet-class>HelloServlet</servlet-class>
</servlet>
```

2. DispatcherServlet Configuration

In old Spring MVC:

```
XML
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
</servlet>
```

Spring Boot removes most of this configuration.

3. Context Path

Application URL base path.

Example:

```
http://localhost:8080/myapp
```

myapp = context path.

4. Port Configuration

Default:

```
8080
```

Spring Boot:

```
properties
server.port=9090
```

5. Application Properties

Very important.

In Spring Boot:

```
properties
server.port=8080
spring.datasource.url=
spring.jpa.hibernate.ddl-auto=update
```

What is WAR File?

WAR = Web Application Archive

Extension:

```
.war
```

Contains:

- JSP
- Servlets
- Classes
- Libraries
- Config files

Used for deploying web applications to external servers like:

- Tomcat
- JBoss
- WebLogic

WAR File Structure

```
myapp.war
├── WEB-INF
│   ├── classes
│   ├── lib
│   └── web.xml
```

How WAR Works

```
WAR File
↓
Deploy into Tomcat
↓
Tomcat runs application
```

What is JAR File?

JAR = Java Archive

Extension:

```
.jar
```

Contains:

- Compiled Java classes
- Libraries
- Metadata

Used for:

- Standalone Java apps
- Spring Boot apps

Spring Boot Uses JAR Mostly

Example:

```
Bash
java -jar app.jar
```

Embedded Tomcat runs internally.

Difference Between WAR and JAR

JAR	WAR
Java Archive	Web Archive
Standalone apps	Web apps
Embedded server possible	Needs external server
Used in Spring Boot	Used in old MVC apps
Easier deployment	Traditional deployment

Most Important Interview Difference

WAR

External Tomcat needed

Deploy manually.

JAR

Embedded Tomcat included

Run directly.

Spring Boot JAR Flow

```
app.jar
  ↓
Embedded Tomcat starts
  ↓
Application runs
```

Traditional Spring MVC WAR Flow

```
app.war
  ↓
Deploy into Tomcat
  ↓
Tomcat runs app
```

Why Spring Boot Became Popular

Because:

- No external server setup
- No WAR deployment complexity
- Faster deployment
- Easier microservices

Important Interview Questions

Servlet Questions

1. What is Servlet?

Java program that handles HTTP requests and responses.

2. What is Servlet lifecycle?

```
init()
service()
destroy()
```

3. Difference between GET and POST?

GET	POST
Data in URL	Data in body
Less secure	More secure

4. What is HttpServlet?

Base servlet class for HTTP handling.

Tomcat Questions

5. What is Tomcat?

Web server and servlet container for running Java web applications.

6. What is default Tomcat port?

8080

7. Difference between Tomcat and Apache HTTP Server?

Tomcat	Apache HTTP
Java apps	Static content
Servlet container	Web server

WAR/JAR Questions

8. Difference between WAR and JAR?

Main answer:

- WAR → external server required
- JAR → standalone executable

9. Why Spring Boot uses JAR?

Because embedded Tomcat is included.

10. Can Spring Boot generate WAR?

Yes.

By changing:

```
XML
<packaging>war</packaging>
```

Important Real-World Configurations

Database Configuration

```
properties
spring.datasource.url=
spring.datasource.username=
spring.datasource.password=
```

Hibernate Configuration

```
properties
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

Logging Configuration

```
properties
```

```
logging.level.root=INFO
```

Profile Configuration

```
properties
spring.profiles.active=dev
```

Used for:

- DEV
- UAT
- PROD

Very important in enterprise projects.

Most Asked Advanced Topics

Since you're working in enterprise Java at Aurionpro, prepare these heavily:

- Servlet vs Spring MVC
- Embedded Tomcat internals
- Filters vs Interceptors
- Context path
- Session management
- Cookie vs Session
- CORS
- CSRF
- Load balancing
- Reverse proxy
- Nginx basics
- API Gateway
- Microservices deployment
- Dockerizing Spring Boot apps
- JVM memory arguments
- Connection pooling

These are extremely common in Spring Boot interviews for 2-5 years experience.

Explain Spring Boot Annotations
 @SpringBootApplication
 @EnableAutoConfiguration

@ComponentScan
 @ConfigurationProperties
 @Component
 @Service
 @Repository
 @Configuration
 @Bean
 @Autowired
 @Qualifier
 @Scope
 @Lazy
 @Primary
 @RequestMapping
 @GetMapping
 @PostMapping
 @RequestParam
 @PathVariable
 @RequestBody
 @ResponseBody
 @RequestHeader
 @ExceptionHandler
 @Controller vs @RestController difference
 @Transactional
 @EnableTransactionManagement

Explain with examples and interview answers where and how it is used and works

Show moreShow less

Spring Boot Annotations Explained

These annotations are heavily used in Spring Boot and are very common in interviews.

1. @SpringBootApplication

Most important annotation.

```

Java
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
  
```

What It Does Internally

Combination of:

```
Java
@Configuration
@EnableAutoConfiguration
@ComponentScan
```

Interview Answer

@SpringBootApplication is the main annotation of Spring Boot that enables auto configuration, component scanning, and configuration support.

2. @EnableAutoConfiguration

Automatically configures beans based on dependencies.

Example:

- If MySQL dependency exists → datasource configured automatically.
- If Spring MVC exists → DispatcherServlet configured.

Internal Working

Uses:

```
META-INF/spring.factories
```

Spring checks dependencies and loads configurations dynamically.

Interview Answer

It enables Spring Boot auto configuration mechanism based on classpath dependencies.

3. @ComponentScan

Scans packages for Spring beans.

```
Java
@ComponentScan("com.example")
```

Detects:

- @Component
- @Service
- @Repository
- @Controller

Interview Point

Without component scanning:

- Beans won't be created.

4. @ConfigurationProperties

Used to map properties file values into Java object.

Example

application.properties

```
properties
app.name=BankingApp
app.version=1.0
```

Java Class

```
Java
@ConfigurationProperties(prefix = "app")
@Component
public class AppConfig {

    private String name;
    private String version;

    // getters setters
}
```

Why Used?

Better than multiple @Value.

Used for:

- External configuration
- Clean config management

5. @Component

Generic Spring bean.

```
Java
@Component
class Engine {
```

```
}
```

Spring automatically creates bean.

When Used?

For generic utility/helper classes.

6. @Service

Specialized component for business logic.

```
Java
@Service
class UserService {
}
```

Why Use Instead of @Component?

Semantics/readability.

Interview answer:

Used for service/business layer classes.

7. @Repository

Used for DAO/database layer.

```
Java
@Repository
class UserRepository {
}
```

Special Feature

Converts DB exceptions into Spring exceptions.

Very important interview point.

8. @Configuration

Marks configuration class.

```
Java
@Configuration
class AppConfig {
}
```

Contains bean definitions.

9. @Bean

Manually creates bean.

```
Java
@Bean
public ObjectMapper mapper() {
    return new ObjectMapper();
}
```

When Used?

- Third-party classes
- Custom bean creation

Difference: @Bean VS @Component

@Bean	@Component
Method-level	Class-level
Manual creation	Auto scanning

10. @Autowired

Automatic dependency injection.

```
Java
@Autowired
private UserService service;
```

Internal Working

Spring:

1. Finds matching bean
2. Uses reflection
3. Injects dependency

Interview Point

Preferred:

Java

Constructor Injection

11. @Qualifier

Used when multiple beans of same type exist.

```
Java
@Autowired
@Qualifier("upiPaymentService")
private PaymentService service;
```

Without It

Exception:

```
NoUniqueBeanDefinitionException
```

12. @Scope

Defines bean scope.

```
Java
@Scope("prototype")
@Component
class Employee {
}
```

Types

Scope	Meaning
singleton	One object
prototype	Multiple objects
request	Per HTTP request
session	Per session

13. @Lazy

Bean created only when needed.

```
Java
@Lazy
@Component
class HeavyService {
}
```


Default Behavior

Spring beans are eager by default.

14. @Primary

Default bean selected when multiple exist.

```
Java
@Primary
@Service
class UpiPaymentService implements PaymentService {
}
```

15. @RequestMapping

Maps URL to method/class.

```
Java
@RequestMapping("/users")
```

Can handle:

- GET
- POST
- PUT
- DELETE

16. @GetMapping

Shortcut for GET request.

```
Java
@GetMapping("/users")
```

Equivalent:

```
Java
@RequestMapping(method = RequestMethod.GET)
```

17. @PostMapping

Handles POST requests.

```
Java
@PostMapping("/save")
```

Used for:

- Create operations

18. @RequestParam

Reads query parameters.

URL:

```
/user?id=10
```

Controller:

```
Java
@GetMapping
public String get(@RequestParam int id)
```

19. @PathVariable

Reads value from URL path.

URL:

```
/user/10
```

Controller:

```
Java
@GetMapping("/user/{id}")
public String get(@PathVariable int id)
```

Difference

@RequestParam	@PathVariable
Query param	URL path
?id=1	/1

20. @RequestBody

Converts JSON → Java object.

JSON:

```
JSON
{
  "name": "Lawrence"
}
```

Controller:

```
Java
@PostMapping
public void save(@RequestBody User user)
```

Uses:

- Jackson library internally

21. @ResponseBody

Converts Java object → JSON response.

```
Java
@ResponseBody
public User getUser() {
    return new User();
}
```

Important Point

@RestController

already contains:

```
Java
@Controller + @ResponseBody
```

22. @RequestHeader

Reads HTTP headers.

```
Java
@GetMapping
public String test(@RequestHeader("Authorization") String token)
```

Used heavily in JWT authentication.

23. @ExceptionHandler

Handles exceptions.

```
Java
@ExceptionHandler(Exception.class)
public String handle(Exception ex) {
    return ex.getMessage();
}
```

Usually Used With

Java
@ControllerAdvice

Global exception handling.

24. @Controller VS @RestController

Very important interview question.

@Controller

Returns view.

```
Java
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }
}
```

Returns:

home.jsp

@RestController

Returns JSON directly.

```
Java
@RestController
public class UserController {

    @GetMapping("/user")
    public User getUser() {
        return new User();
    }
}
```

Difference Table

@Controller	@RestController
Returns View	Returns JSON
MVC apps	REST APIs
JSP/HTML	Microservices

25. @Transactional

Very important interview topic.

Used for database transaction management.

```
Java
@Transactional
public void transferMoney() {
}
```

What It Does

If exception occurs:

Rollback transaction

Otherwise:

Commit transaction

Internal Working

Spring creates proxy around method.

Uses:

- AOP
- Proxy pattern

Very important interview point.

Example

```
Java
@Transactional
public void transfer() {
    debit();
    credit();
    // if error → rollback
}
```

Transaction Propagation Types

Common ones:

Type	Meaning
REQUIRED	Default

Type	Meaning
REQUIRES_NEW	New transaction
SUPPORTS	Use existing if available

26. @EnableTransactionManagement

Enables transaction support.

```
Java
@EnableTransactionManagement
@Configuration
class Config {
}
```

Usually auto-enabled in Spring Boot.

Internal Working

Enables:

- Transaction proxies
- AOP interceptors

Most Important Interview Questions

1. Difference between @Component, @Service, @Repository?

All create beans.

Used semantically for different layers.

2. Difference between @Controller and @RestController?

@Controller	@RestController
View	JSON

3. Difference between @RequestParam and @PathVariable?

| Query param | URL param |

4. How does @Autowired work internally?

Spring:

- finds bean
- uses reflection
- injects dependency

5. How does @Transactional work internally?

Using:

- AOP
- Proxy pattern

6. Difference between @Bean and @Component?

| Manual | Automatic |

7. What happens if multiple beans exist?

Use:

- @Primary
- @Qualifier

8. Why Constructor Injection preferred?

- Immutable
- Easier testing
- Mandatory dependencies

Most Asked Advanced Topics

For enterprise interviews at companies like Aurionpro:

Prepare:

- Spring AOP
- Proxy pattern
- Bean lifecycle
- Spring Security annotations
- Transaction isolation levels
- LazyInitializationException
- JPA annotations
- Hibernate caching
- Exception handling
- Filters vs Interceptors
- Spring Boot auto configuration internals
- Circular dependency handling

These are heavily asked in 2–5 year Spring Boot interviews.

Explain @Transactional more

What is @Transactional?

In Spring Framework, `@Transactional` is used to manage database transactions automatically.

It ensures:

- Data consistency
- Atomicity
- Rollback on failure

Simple Definition

`@Transactional` ensures that a group of database operations either all succeed or all fail together.

Real Life Banking Example

Suppose:

Transfer ₹1000
from Account A → Account B

Steps:

1. Debit Account A

2. Credit Account B

Problem Without Transaction

Imagine:

- Amount debited from A
- Server crashes before crediting B

Money lost.

Solution Using @Transactional

If any step fails:

Entire transaction rollback

Database restored to previous state.

Example

```
Java
@Service
public class BankService {

    @Transactional
    public void transferMoney() {

        debitAccount();

        creditAccount();

    }
}
```

What Happens Internally?

Spring:

1. Creates proxy object
2. Starts transaction before method
3. Executes method
4. If success → COMMIT
5. If exception → ROLLBACK

Internal Flow

```
Client
↓
Spring Proxy
↓
Transaction Started
↓
Method Executes
↓
Commit / Rollback
```

Very Important Interview Point

@Transactional works using:

- AOP (Aspect Oriented Programming)
- Proxy Pattern

Most asked answer.

Example with Failure

```
Java
@Transactional
public void transfer() {
    debit();
    int x = 10 / 0;
    credit();
}
```

What Happens?

```
ArithmeticException occurs
↓
Spring rolls back transaction
↓
debit() also reverted
```

Database remains consistent.

Transaction Management Without Spring

Without Spring:

```
Java
connection.setAutoCommit(false);
try {
    // queries
    connection.commit();
} catch (Exception e) {
```

```
connection.rollback();
}
```

Huge boilerplate code.

Spring simplifies this using `@Transactional`.

Important ACID Properties

Transactions follow ACID.

A → Atomicity

All operations succeed or fail together.

C → Consistency

Database remains valid.

I → Isolation

Transactions don't interfere.

D → Durability

Committed data persists permanently.

Where `@Transactional` Is Used

Mostly:

- Service layer

Example:

```
Java
@Service
public class PaymentService {
}
```

Why Not Controller Layer?

Best practice:

- Business logic belongs in service layer

- Transaction management should stay there

Rollback Rules

Default Behavior

Spring rolls back only for:

Unchecked Exceptions
(`RuntimeException`)

Examples:

- `NullPointerException`
- `ArithmeticException`

Checked Exceptions Problem

```
Java
throw new IOException();
```

By default:

NO rollback

Very important interview question.

Rollback for Checked Exceptions

```
Java
@Transactional(rollbackFor = Exception.class)
```

Now rollback happens for checked exceptions too.

Example

```
Java
@Transactional(rollbackFor = Exception.class)
public void save() throws Exception {
    throw new IOException();
}
```

Propagation in Transactions

Very important advanced topic.

Defines:

How transactions behave when one transactional method calls another.

1. REQUIRED (Default)

```
Java
@Transactional(propagation = Propagation.REQUIRED)
```

Behavior:

- Use existing transaction
- Else create new

Most used.

Example

```
Method A starts TX
↓
Method B joins same TX
```

2. REQUIRES_NEW

```
Java
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

Always creates new transaction.

Even if existing transaction exists.

Example

```
Main transaction running
↓
Audit logging creates separate transaction
```

Useful for:

- Audit logs
 - Notifications
-

3. SUPPORTS

```
Java
@Transactional(propagation = Propagation.SUPPORTS)
```

Behavior:

- Join transaction if exists
- Else run without transaction

4. MANDATORY

Requires existing transaction.

If none:

Exception thrown

5. NEVER

Should NOT run inside transaction.

Isolation Levels

Controls concurrent transaction behavior.

Very important for banking projects.

Common Problems

Problem	Meaning
Dirty Read	Reading uncommitted data
Non-repeatable Read	Data changes during transaction
Phantom Read	New rows appear

Isolation Levels

Isolation	Prevents
READ_UNCOMMITTED	Nothing
READ_COMMITTED	Dirty reads
REPEATABLE_READ	Non-repeatable reads
SERIALIZABLE	All issues

Example

```
Java
@Transactional(
    isolation = Isolation.READ_COMMITTED
)
```

Transaction Timeout

```
Java
@Transactional(timeout = 5)
```

If method exceeds 5 seconds:

Transaction rolled back

Read Only Transaction

Optimization for SELECT queries.

```
Java
@Transactional(readOnly = true)
```

Improves performance.

Very Important Proxy Concept

Most asked advanced interview question.

How Spring Applies Transaction?

Spring creates:

Proxy Object

Instead of actual object.

Flow

```
Client
↓
Proxy
↓
Start Transaction
↓
Original Method
↓
Commit/Rollback
```

Internal Classes Used

Spring internally uses:

- TransactionInterceptor
- PlatformTransactionManager
- AOP proxies

Self Invocation Problem

Very commonly asked.

Example

```
Java
@Service
class UserService {

    public void methodA() {
        methodB();
    }

    @Transactional
    public void methodB() {
    }
}
```

Problem

methodB() transaction may NOT work.

Why?

Internal method call bypasses proxy

Very important interview point.

Solution

Call method from another bean.

Types of Transaction Managers

Manager	Used For
DataSourceTransactionManager	JDBC
JpaTransactionManager	JPA/Hibernate
JtaTransactionManager	Distributed TX

Common Interview Questions

1. What is @Transactional?

Used for automatic transaction management in Spring.

2. How does @Transactional work internally?

Using AOP and proxy pattern.

3. What happens if exception occurs?

Spring rolls back transaction.

4. Does rollback happen for checked exceptions?

Default:

NO

Need:

```
Java  
rollbackFor = Exception.class
```

5. Why use transactions?

To maintain:

- Consistency
 - Atomicity
-

6. Why should transactions be in service layer?

Because service layer contains business logic.

7. What is propagation?

Defines transaction behavior between nested methods.

8. What is isolation level?

Controls concurrent transaction issues.

9. What is self-invocation issue?

Internal method calls bypass Spring proxy.

10. What is readOnly transaction?

Optimization for read operations.

Real Interview Scenario Questions

Scenario 1

Money debited but not credited. Why?

Possible reason:

- Missing transaction management.
-

Scenario 2

@Transactional not working. Why?

Possible reasons:

- Method is private
 - Self invocation
 - Bean not managed by Spring
 - Exception swallowed
-

Scenario 3

Why transaction not rolled back?

Because:

- Checked exception
- Catch block swallowed exception

Best Interview Answer

`@Transactional` provides declarative transaction management in Spring. It works using AOP proxies. Spring starts transaction before method execution and commits or rolls back based on method outcome.

Spring Hibernate and JPA Explain Concepts and topics and Interview Questions asked

Spring, Hibernate, and JPA — Complete Explanation

This is one of the most important backend interview topics for Java developers.

First Understand Relationship

```

Spring Boot
  ↓
Spring Data JPA
  ↓
JPA Specification
  ↓
Hibernate Implementation
  ↓
Database
  
```

1. What is JPA?

Jakarta Persistence (earlier Java Persistence API) is:

A specification for ORM in Java.

Important:

JPA is NOT a framework

It only defines rules/interfaces.

What JPA Provides

JPA defines:

- Entity mapping
- CRUD operations
- Relationships
- Query mechanisms

Example JPA Interfaces

```
Java
EntityManager
PersistenceContext
Query
```

Interview Answer

JPA is a specification for ORM in Java. It defines standards for mapping Java objects to database tables.

2. What is Hibernate?

Hibernate is:

An implementation of JPA.

Hibernate actually performs:

- SQL generation
- DB communication
- Object mapping
- Caching
- Transactions

Important Point

```
JPA = Specification
Hibernate = Implementation
```

Very commonly asked.

Example

When you write:

```
Java
userRepository.save(user);
```

Internally:

```
Spring Data JPA
→ JPA
→ Hibernate
→ SQL Query
```

3. What is ORM?

ORM = Object Relational Mapping

Converts:

```
Java Objects ↔ Database Tables
```

Example

Java Class

```
Java
class User {
    Long id;
    String name;
}
```

Database Table

```
USER
----
ID
NAME
```

Hibernate maps both automatically.

Why ORM?

Without ORM:

- Manual SQL
- Manual ResultSet handling

ORM simplifies development.

4. What is Spring Data JPA?

Spring Data JPA simplifies database operations.

Provides:

- Repository abstraction
- Automatic query generation
- Pagination
- Sorting

Example

```
Java
@Repository
public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

No SQL needed for basic CRUD.

CRUD Operations

```
Java
save()
findById()
findAll()
deleteById()
```

Automatically available.

Important JPA Annotations

@Entity

Marks class as DB entity.

```
Java
@Entity
class User {
}
```

@Table

Maps table name.

```
Java
@Table(name = "users")
```

@Id

Primary key.

```
Java
@Id
private Long id;
```

@GeneratedValue

Auto ID generation.

```
Java
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

Generation Strategies

Strategy	Meaning
AUTO	Auto choose
IDENTITY	DB auto increment
SEQUENCE	Sequence
TABLE	Table-based

@Column

Maps column.

```
Java
@Column(name = "user_name")
```

@Transient

Ignore field.

```
Java
@Transient
private String temp;
```

Not stored in DB.

Relationships in Hibernate/JPA

Very important interview topic.

1. One-to-One

Example:

- User ↔ Passport

```
Java
@OneToOne
private Passport passport;
```

2. One-to-Many

Example:

- Customer → Orders

```
Java
@OneToMany
private List<Order> orders;
```

3. Many-to-One

Example:

- Many Employees → One Department

```
Java
@ManyToOne
private Department department;
```

4. Many-to-Many

Example:

- Student ↔ Course

```
Java
@ManyToMany
private List<Course> courses;
```

Fetch Types

Very important.

EAGER Fetch

Loads immediately.


```
Java
@OneToMany(fetch = FetchType.EAGER)
```

LAZY Fetch

Loads only when accessed.

```
Java
@OneToMany(fetch = FetchType.LAZY)
```

Default Fetch Types

Relation	Default
OneToMany	LAZY
ManyToOne	EAGER

Very Important Interview Topic

LazyInitializationException

Occurs when:

```
Lazy object accessed outside session
```

Very common in interviews.

Example

```
Java
user.getOrders();
```

after session closed.

Solutions

- Use JOIN FETCH
- DTO mapping
- OpenSessionInView
- Transactional scope

Cascade Types

Controls child operations.

Example

Java

```
@OneToMany(cascade = CascadeType.ALL)
```

Types

Cascade	Meaning
PERSIST	Save child
REMOVE	Delete child
MERGE	Update child
ALL	Everything

Hibernate Session

Very important.

What is Session?

Represents:

Connection between app and DB

Used for:

- Save
- Update
- Delete
- Fetch

Entity States

Very commonly asked.

1. Transient

Object not connected to DB.

```
Java
User u = new User();
```

2. Persistent

Managed by Hibernate session.

```
Java
session.save(u);
```

3. Detached

Session closed but object exists.

4. Removed

Marked for deletion.

First Level Cache

Hibernate session cache.

Default enabled.

Scope:

```
Per Session
```

Second Level Cache

Shared cache across sessions.

Needs configuration.

Tools:

- EhCache
 - Redis
-

JPQL

Java Persistence Query Language.

Works on entities, not tables.

Example

Java

```
SELECT u FROM User u
```

Not:

SQL

```
SELECT * FROM users
```

Native Query

Real SQL query.

Java

```
@Query(value = "SELECT * FROM users",
      nativeQuery = true)
```

Derived Query Methods

Spring generates queries automatically.

Java

```
findByEmail(String email)
```

Very important feature.

Hibernate vs JDBC

Hibernate	JDBC
ORM	Low level
Less code	More boilerplate
Automatic mapping	Manual mapping
DB independent	DB specific

JPA vs Hibernate

JPA	Hibernate
Specification	Implementation
Defines rules	Executes rules

Most asked interview question.

Spring Data JPA vs Hibernate

Spring Data JPA	Hibernate
Repository abstraction	ORM engine
Simplifies queries	Actual implementation

N+1 Problem

Very important advanced topic.

Problem

1 query for parent
N queries for children

Performance issue.

Solution

Use:

```
Java
JOIN FETCH
@EntityGraph
```

Example

```
Java
SELECT u FROM User u
JOIN FETCH u.orders
```

Dirty Checking

Very commonly asked.

Hibernate automatically detects changes.

Example:

```
Java
user.setName("Lawrence");
```

No explicit update needed.

Hibernate updates automatically at transaction commit.

Interview Questions

Basic Questions

1. What is JPA?

JPA is a specification for ORM in Java.

2. What is Hibernate?

Hibernate is an ORM framework and implementation of JPA.

3. Difference between JPA and Hibernate?

JPA	Hibernate
Specification	Implementation

4. What is ORM?

Object ↔ Table mapping.

5. What is Entity?

Java class mapped to DB table.

Intermediate Questions

6. Difference between `save()` and `saveAndFlush()`?

<code>save</code>	<code>saveAndFlush</code>
Delayed DB sync	Immediate flush

7. What is `LazyInitializationException`?

Accessing lazy object after session closed.

8. What is CascadeType?

Defines child entity operations.

9. Difference between EAGER and LAZY?

EAGER	LAZY
Immediate load	On-demand

10. What is Hibernate Session?

DB interaction context.

Advanced Questions

11. What is N+1 problem?

Multiple unnecessary queries issue.

12. What is dirty checking?

Hibernate auto detects modified entities.

13. First level vs Second level cache?

First	Second
Session scope	Application scope

14. Difference between get() and load()?

get	load
Immediate DB hit	Proxy object

15. What is flushing?

Synchronizing persistence context with DB.

16. Difference between `persist()` and `merge()`?

persist	merge
New entity	Detached entity

Most Asked Real Project Questions

Since you work on enterprise apps at Aurionpro, prepare these deeply:

- LazyInitializationException
- N+1 query problem
- Transaction management
- Dirty checking
- Fetch joins
- Entity lifecycle
- Optimistic vs pessimistic locking
- Indexing
- Batch processing
- Query optimization
- Pagination
- Connection pooling
- Hibernate caching
- Composite keys
- DTO projections

These are extremely common in 2–5 year Spring Boot interviews.

custom JPA queries how to write

Custom JPA Queries in Spring Boot

In Spring Data JPA, custom queries are written when:

- Derived methods are not enough
- Complex joins needed
- Performance optimization needed
- Native SQL required

Ways to Write Custom Queries

Method	Used For
Derived Query Methods	Simple queries
@Query with JPQL	Entity-based queries
Native SQL Query	Real SQL
Named Queries	Reusable queries
Criteria API	Dynamic queries
Specification API	Dynamic filtering

1. Derived Query Methods

Spring automatically generates query from method name.

Example

```

Java
@Repository
public interface UserRepository
    extends JpaRepository<User, Long> {

    User findByEmail(String email);

    List<User> findByStatus(String status);

    List<User> findByAgeGreaterThan(int age);
}

```

Internally Generated Query

```

SQL
SELECT * FROM users WHERE email = ?

```

Common Keywords

Keyword	Meaning
findBy	Fetch
And	AND condition
Or	OR condition
Between	Range
LessThan	<
GreaterThan	>
Like	LIKE query
In	IN clause
OrderBy	Sorting

Examples

Java

```

findByNameAndStatus()
findBySalaryGreaterThan()
findByAgeBetween()
findByNameLike()
findByDepartmentIn()

```

2. @Query with JPQL

Very important interview topic.

JPQL works on:

Entities and fields
NOT tables and columns

Example

Java

```

@Query("SELECT u FROM User u WHERE u.email = :email")
User getEmail(@Param("email") String email);

```

Important Point

Here:

Java
User

is:

Entity name

NOT table name.

Another Example

```
Java
@Query("SELECT u FROM User u WHERE u.salary > :salary")
List<User> getHighSalaryUsers(
    @Param("salary") double salary);
```

3. Native SQL Query

Uses actual SQL.

Example

```
Java
@Query(
    value = "SELECT * FROM users WHERE email = :email",
    nativeQuery = true
)
User findUser(@Param("email") String email);
```

When Native Query is Used

Used for:

- Complex joins
- DB-specific queries
- Performance tuning
- Stored procedures

Difference Between JPQL and Native Query

JPQL	Native SQL
Entity-based	Table-based
DB independent	DB specific
Portable	Faster sometimes

4. Named Queries

Reusable queries.

Example

Entity

```
Java
@NamedQuery(
    name = "User.findByEmail",
    query = "SELECT u FROM User u WHERE u.email=:email"
)
@Entity
class User {
}
```

Repository

```
Java
@Query(name = "User.findByEmail")
User getByEmail(@Param("email") String email);
```

5. Pagination Queries

Very commonly used.

Example

```
Java
Page<User> findByStatus(
    String status,
    Pageable pageable);
```

Usage

```
Java
PageRequest.of(0, 10);
```

Generated Query

```
SQL
LIMIT 10 OFFSET 0
```

6. Sorting Queries

Example

Java

```
List<User> findByStatus(
    String status,
    Sort sort
);
```

Usage

Java

```
Sort.by("name").ascending()
```

7. Update Queries

Very important.

Example

Java

```
@Modifying
@Transactional
@Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
int updateStatus(
    @Param("status") String status,
    @Param("id") Long id);
```

Important Interview Point

For UPDATE/DELETE:

Must use:

Java

```
@Modifying
```

Why?

Because:

Default queries are SELECT only

8. Delete Query

Java

```
@Modifying
@Transactional
@Query("DELETE FROM User u WHERE u.status = :status")
int deleteUsers(@Param("status") String status);
```

9. Join Queries

Very commonly asked.

Example

```
Java
@Query("""
SELECT u
FROM User u
JOIN u.orders o
WHERE o.amount > :amount
""")
List<User> getUsers(double amount);
```

Fetch Join

Important for solving:

N+1 Problem

Example

```
Java
@Query("""
SELECT u
FROM User u
JOIN FETCH u.orders
""")
List<User> getUsers();
```

Difference

JOIN	JOIN FETCH
Normal join	Loads child entities immediately

10. Projection Queries

Fetch only needed fields.

Better performance.

DTO Projection

```
Java
@Query("""
SELECT new com.dto.UserDTO(u.id, u.name)
FROM User u
""")
List<UserDTO> getUsers();
```

Interface Projection

```
Java
public interface UserView {
    String getName();
}
```

Repository

```
Java
List<UserView> findByStatus(String status);
```

11. Dynamic Queries Using Specification

Very important for enterprise apps.

Used for:

- Search APIs
- Filter APIs

Example

```
Java
public class UserSpecification {
    public static Specification<User>
        hasStatus(String status) {
        return (root, query, cb) ->
            cb.equal(root.get("status"), status);
    }
}
```

Repository

```
Java
extends JpaSpecificationExecutor<User>
```

Usage

```
Java
repo.findAll(hasStatus("ACTIVE"));
```

12. Criteria API

Dynamic query building.

Very verbose.

Used less nowadays.

Example

```
Java
CriteriaBuilder cb =
    entityManager.getCriteriaBuilder();
```

13. Stored Procedure Queries

Very common in banking projects.

Example

```
Java
@Procedure(name = "get_user")
String getUser();
```

Alternative

```
Java
@Query(value = "CALL get_user()", nativeQuery = true)
```

Query Parameters

Named Parameters

```
Java
:email
```

Used with:

```
Java
@Param
```

Positional Parameters

```
Java
?1
```

Example:

```
Java
@Query("SELECT u FROM User u WHERE u.email = ?1")
```


Which Is Better?

Named Parameters

More readable.

Query Optimization Tips

Very important in interviews.

Use Pagination

Avoid:

```
Java  
findAll()
```

on huge tables.

Use DTO Projections

Fetch only needed fields.

Avoid N+1 Problem

Use:

```
Java  
JOIN FETCH
```

Use Indexes

Index frequently searched columns.

Use Native Query Carefully

Only when necessary.

Common Interview Questions

1. Difference between JPQL and Native Query?

JPQL	Native
Entity based	SQL based

2. Why use @Modifying?

Required for:

- UPDATE
- DELETE

queries.

3. Difference between findById() and custom query?

`findById()`:

- auto-generated

Custom query:

- manual logic

4. What is JPQL?

Query language working on entities instead of tables.

5. What is N+1 problem?

One parent query + multiple child queries.

6. How to solve N+1 issue?

Using:

```
Java
JOIN FETCH
```

7. Difference between LAZY and EAGER?

LAZY	EAGER
On-demand	Immediate

8. Why DTO projection?

Better performance.

9. Difference between save() and saveAndFlush()?

save	saveAndFlush
Delayed flush	Immediate flush

10. What is Specification API?

Dynamic query builder.

Most Asked Real Project Questions

Since you're working on enterprise applications at Aurionpro, focus heavily on:

- Dynamic filtering APIs
- Pagination + sorting
- Native query optimization
- DTO projections
- JOIN FETCH
- Batch updates
- Stored procedures
- Transaction handling
- LazyInitializationException
- Query performance tuning
- Indexing
- Composite keys
- Entity graphs

- Optimistic locking

These are extremely common in Spring Boot + JPA interviews for 2–5 years experience.

Spring Security, JWT etc all concepts need to know
Most asked interview Questions

Spring Security + JWT Complete Guide

Spring Security is one of the most important topics in Java backend interviews.

Especially for:

- REST APIs
- Banking apps
- Microservices
- Enterprise applications

What is Spring Security?

Spring Security is a framework used for authentication and authorization in Java applications.

Provides:

- Login authentication
- Role-based authorization
- JWT authentication
- OAuth2
- CSRF protection
- Session management
- Password encryption

Authentication vs Authorization

Very commonly asked.

Authentication	Authorization
Who are you?	What can you access?
Login verification	Permission checking

Example

Authentication

Username + Password verification

Authorization

ADMIN can delete users
USER cannot

Spring Security Flow

```

Request
↓
Security Filter Chain
↓
Authentication
↓
Authorization
↓
Controller

```

Important Interview Point

Spring Security works mainly using:

Filters

Very important answer.

What is Filter Chain?

Request passes through multiple filters.

Examples:

- Authentication filter
- JWT filter
- CSRF filter
- Authorization filter

Common Security Filters

Filter	Purpose
UsernamePasswordAuthenticationFilter	Login auth
BasicAuthenticationFilter	Basic auth
OncePerRequestFilter	JWT validation

Filter	Purpose
CsrfFilter	CSRF protection

What is JWT?

JSON Web Token is:

A compact token used for stateless authentication.

Why JWT?

Instead of storing sessions on server:

JWT stores user info inside token

Server becomes stateless.

Very important for:

- Microservices
- REST APIs

JWT Structure

Header.Payload.Signature

Example:

eyJhbGciOi0i...

1. Header

Contains:

```
JSON
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Contains user data.

JSON

```
{
  "username": "lawrence",
  "role": "ADMIN"
}
```

Called:

Claims

3. Signature

Used to verify token integrity.

Generated using:

- Secret key
- Algorithm

JWT Authentication Flow

Very important interview question.

```
Client Login
↓
Username + Password
↓
Spring Security verifies
↓
JWT generated
↓
Client stores token
↓
Client sends token in every request
↓
Server validates token
```

Authorization Header

Authorization: Bearer eyJhbGci...

Stateless Authentication

JWT is:

STATELESS

Server does NOT store session.

Session vs JWT

Session	JWT
Stateful	Stateless
Server memory used	No server storage
Better logout handling	Better scalability
Traditional apps	Microservices/APIs

Important Spring Security Components

1. SecurityFilterChain

Very important in Spring Boot 3.

Example

```
Java
@Bean
public SecurityFilterChain securityFilterChain(
    HttpSecurity http) throws Exception {

    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/admin/**")
            .hasRole("ADMIN")
            .anyRequest()
            .authenticated()
        );

    return http.build();
}
```

What It Does

- Disables CSRF
- Protects APIs
- Applies role authorization

2. UserDetailsService

Loads user from DB.

Example

```
Java
```



```
@Service
public class CustomUserDetailsService
    implements UserDetailsService {

    @Override
    public UserDetails loadUserByUsername(
        String username) {

        return userRepository.findByUsername(username);
    }
}
```

3. UserDetails

Represents authenticated user.

Contains:

- username
- password
- roles

4. PasswordEncoder

Passwords should NEVER be stored plain text.

Used:

```
Java
BCryptPasswordEncoder
```

Example

```
Java
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Why BCrypt?

Because:

- Password hashing
- Salt internally added
- Secure against rainbow attacks

5. AuthenticationManager

Responsible for authentication.

Flow

```
Username
↓
Password verification
↓
Authentication success/failure
```

6. Authentication Object

Represents logged-in user.

```
Java
Authentication auth =
    SecurityContextHolder
        .getContext()
        .getAuthentication();
```

SecurityContextHolder

Stores authenticated user details.

Very important interview point.

JWT Generation Example

```
Java
String token =
    Jwts.builder()
        .setSubject(username)
        .signWith(secretKey)
        .compact();
```

JWT Validation Example

```
Java
Jwts.parserBuilder()
    .setSigningKey(secretKey)
    .build()
    .parseClaimsJws(token);
```

JWT Filter

Very important interview topic.

Purpose

Every request:

1. Read JWT token
2. Validate token
3. Set authentication

Example

```
Java
public class JwtFilter
    extends OncePerRequestFilter {
}
```

Why OncePerRequestFilter?

Ensures filter executes:

Once per request

Role Based Authorization

Example

```
Java
.hasRole("ADMIN")
```

or

```
Java
.hasAuthority("READ_PRIVILEGE")
```

Difference

Role	Authority
High-level	Fine-grained permission

Method-Level Security

Example

```
Java
@PreAuthorize("hasRole('ADMIN')")
```

Enables

```
Java
@EnableMethodSecurity
```

CSRF

Very commonly asked.

What is CSRF?

Cross Site Request Forgery attack.

Example

User logged into bank.

Attacker tricks browser into sending:

```
transfer money request
```

without user knowing.

Why Disabled in JWT APIs?

Because JWT APIs are:

```
Stateless
```

Usually:

```
Java
csrf.disable()
```

CORS

Very important frontend-backend topic.

What is CORS?

Cross-Origin Resource Sharing.

Allows frontend:

```
localhost:3000
```

to access backend:

```
localhost:8080
```

Example

```
Java
@CrossOrigin(origins = "**")
```

Authentication Types

Type	Usage
Basic Auth	Simple apps
Session Auth	Traditional MVC
JWT Auth	REST APIs
OAuth2	Google login
LDAP	Enterprise login

OAuth2

Used for:

- Google login
- Facebook login
- GitHub login

Difference Between JWT and OAuth2

JWT	OAuth2
Token format	Authorization framework

Exception Handling in Security

AuthenticationException

Invalid login.

AccessDeniedException

User lacks permission.

Common HTTP Status Codes

Code	Meaning
401	Unauthorized
403	Forbidden

401 vs 403

Very commonly asked.

401	403
Not authenticated	No permission

Security Best Practices

Very important in interviews.

Use Password Hashing

Never plain passwords.

Use HTTPS

Encrypt communication.

Token Expiry

JWT should expire.

Refresh Tokens

Used to renew access token.

Avoid Storing Sensitive Data in JWT

JWT payload is visible after decoding.

Important Interview Questions

Basic Questions

1. What is Spring Security?

Framework for authentication and authorization.

2. Authentication vs Authorization?

| Authentication | Authorization |

3. What is JWT?

Stateless authentication token.

4. Structure of JWT?

```
Header.Payload.Signature
```

5. Why JWT preferred in microservices?

Because:

```
Stateless and scalable
```

Intermediate Questions

6. How JWT works internally?

1. Login
 2. Generate token
 3. Client stores token
 4. Token validated for every request
-

7. What is SecurityFilterChain?

Defines security rules and filters.

8. What is UserDetailsService?

Loads user from DB.

9. Why use BCrypt?

Secure password hashing.

10. What is OncePerRequestFilter?

JWT filter executed once/request.

Advanced Questions

11. How does Spring Security work internally?

Using:

- Filter chain
 - AuthenticationManager
 - SecurityContextHolder
-

12. Difference between 401 and 403?

| 401 | 403 |

13. Why disable CSRF in JWT?

Because JWT APIs are stateless.

14. What is Refresh Token?

Used to generate new access token.

15. Difference between Role and Authority?

| Role | Authority |

16. What is SecurityContextHolder?

Stores authenticated user.

17. How does JWT validation happen?

Using:

- Signature verification
 - Expiry validation
-

18. Can JWT be revoked?

Not directly easy.

Usually:

- blacklist
 - short expiry
 - refresh tokens
-

Most Asked Real Project Questions

Since you work on enterprise banking-style applications at Aurionpro, prepare heavily:

- JWT authentication flow
- Filter chain internals
- OAuth2 basics
- Stateless vs stateful auth
- CSRF vs CORS
- Refresh tokens
- Role-based access

- Method security
- Password encryption
- Security vulnerabilities
- API Gateway security
- Microservices authentication
- Token expiration handling
- Spring Security 6 changes
- AuthenticationManager internals

These are extremely common in Spring Boot interviews for 2-5 years experience.

CSRF vs CORS

CSRF vs CORS

Very commonly asked interview topic in Spring Security and frontend-backend development. Many developers confuse them.

Main Difference

CSRF	CORS
Security attack	Browser security policy
Protects user session	Controls cross-origin access
Related to authentication	Related to browser restrictions

1. What is CSRF?

Cross-site request forgery

CSRF means:

An attacker tricks a logged-in user's browser into sending unwanted requests.

Real Example

Suppose user logged into:

bank.com

Session cookie stored in browser.

Now user visits malicious site:

```
hack.com
```

That site secretly sends:

```
POST /transferMoney
```

Browser automatically sends bank session cookie.

Result:

```
Money transferred without user knowing
```

Why CSRF Happens

Because browsers automatically send:

```
Cookies/session IDs
```

with requests.

CSRF Protection

Server generates:

```
CSRF Token
```

Every sensitive request must include token.

Example

```
HTML
<input type="hidden"
      name="csrf"
      value="abc123">
```

Spring Security CSRF

Enabled by default for:

```
Session-based applications
```

Disable CSRF in REST APIs

Common in JWT APIs:

```
Java
```

```
http.csrf(csrf -> csrf.disable());
```

Why Disabled for JWT?

Because JWT authentication is:

Stateless

No session cookie automatically sent.

Very important interview point.

2. What is CORS?

Cross-origin resource sharing

CORS means:

Browser security mechanism that controls requests between different origins.

What is Origin?

Origin =

Protocol + Domain + Port

Example

URL	Origin
http://localhost:3000	Different
http://localhost:8080	Different

Ports differ → origins differ.

CORS Problem Example

Frontend:

localhost:3000

Backend:

localhost:8080

Browser blocks request unless backend allows it.

Browser Error

Access-Control-Allow-Origin missing

Very common.

Why CORS Exists

To prevent malicious websites from reading data from another domain.

CORS Configuration in Spring Boot

Using @CrossOrigin

```
Java
@CrossOrigin(origins = "http://localhost:3000")
@RestController
class UserController {
}
```

Global CORS Configuration

```
Java
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {

        return new WebMvcConfigurer() {

            @Override
            public void addCorsMappings(
                CorsRegistry registry) {

                registry.addMapping("/**")
                    .allowedOrigins("*")
                    .allowedMethods("*");
            }
        };
    }
}
```

Preflight Request

Very important interview question.

Browser sends:

OPTIONS request

before actual request.

Checks:

- Allowed methods
- Allowed headers
- Permissions

Example

```
OPTIONS /users
```

Important CORS Headers

Header	Purpose
Access-Control-Allow-Origin	Allowed origins
Access-Control-Allow-Methods	Allowed methods
Access-Control-Allow-Headers	Allowed headers

CSRF vs CORS Full Difference

Feature	CSRF	CORS
Type	Attack	Browser policy
Purpose	Prevent fake requests	Control cross-origin requests
Related To	Authentication/session	Browser-origin restrictions
Uses Tokens?	Yes	No
Browser Blocks?	No	Yes
Server Side?	Mostly	Both browser + server
Common in	Session auth	Frontend-backend APIs

Key Interview Point

CSRF

Protects:

```
User from malicious requests
```

CORS

Protects:

Browser from unauthorized cross-origin access

Why JWT APIs Usually Disable CSRF

Very commonly asked.

Because:

- JWT stored in header
- Browser does NOT auto-send JWT
- No session cookie dependency

Hence:

```
Java
csrf.disable()
```

Important Scenario

Session-Based App

Needs:

CSRF Protection

JWT REST API

Usually:

```
CSRF disabled
CORS enabled
```

Very important real-world answer.

Common Interview Questions

1. What is CSRF?

Attack where malicious site tricks browser into sending authenticated requests.

2. What is CORS?

Browser security policy controlling cross-origin requests.

3. Difference between CSRF and CORS?

CSRF	CORS
Attack	Browser policy

4. Why disable CSRF in JWT?

Because JWT authentication is stateless.

5. What is preflight request?

Browser sends:

OPTIONS

request before actual request.

6. What is Same-Origin Policy?

Browser policy restricting cross-origin access.

CORS relaxes this policy.

7. Why browser blocks frontend-backend requests?

Different origins.

Need proper CORS configuration.

8. What is origin?

Protocol + Domain + Port

Most Asked Real-World Questions

Since you work on enterprise applications at Aurionpro, expect these:

- Why JWT APIs disable CSRF?
- Why CORS issue occurs in React + Spring Boot?
- How preflight request works?
- Difference between cookies and JWT?
- Why browser blocks APIs?
- How API Gateway handles CORS?
- How CSRF token works internally?
- What are SameSite cookies?
- Difference between Authorization header and Cookies?

These are very common in Spring Boot + Angular/React interviews.

CORS vs CQRS why it is required how it is handled from backend

You're mixing two completely different concepts:

- **CORS** → Web/browser security concept
- **CQRS** → Software architecture pattern

They are unrelated but both commonly asked in backend interviews.

1. What is CORS?

Cross-origin resource sharing

CORS is:

A browser security mechanism that controls whether frontend applications can access backend APIs from another origin.

Why CORS is Required

Browser follows:

Same-Origin Policy

Meaning:

Frontend can call backend only if:

- protocol same
- domain same
- port same

Example

Frontend:

```
http://localhost:3000
```

Backend:

```
http://localhost:8080
```

Ports differ → browser blocks request.

Error Example

```
Access to fetch has been blocked by CORS policy
```

Very common.

How Backend Handles CORS

Backend sends special headers:

```
Access-Control-Allow-Origin
```

Browser checks these headers.

If allowed:

```
Request succeeds
```

Spring Boot CORS Handling

Method 1 — @CrossOrigin

```
Java
@CrossOrigin(origins = "http://localhost:3000")
@RestController
public class UserController {
}
```

Method 2 — Global Configuration

```
Java
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
```

```
return new WebMvcConfigurer() {
    @Override
    public void addCorsMappings(
        CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("*")
            .allowedHeaders("*");
    }
};
```

Important CORS Headers

Header	Purpose
Access-Control-Allow-Origin	Allowed frontend
Access-Control-Allow-Methods	Allowed methods
Access-Control-Allow-Headers	Allowed headers

Preflight Request

Very important interview topic.

Browser sends:

OPTIONS request

before actual request.

Checks:

- allowed methods
- allowed headers
- security rules

Interview Answer for CORS

CORS is a browser security mechanism that allows or restricts cross-origin API calls. Backend handles it by sending appropriate CORS headers.

2. What is CQRS?

Command Query Responsibility Segregation

CQRS means:

Separating READ operations and WRITE operations into different models/services.

Full Form

C → Command
Q → Query
R → Responsibility
S → Segregation

Why CQRS is Required

In large applications:

- Reads are huge
- Writes are complex

Single model becomes bottleneck.

CQRS separates them.

Traditional Architecture

Single Service
↓
Read + Write both handled

CQRS Architecture

WRITE SIDE (Commands)
↓
Database
↓
READ SIDE (Queries)

Separate handling.

Command Side

Handles:

- Insert
- Update
- Delete

Examples:

Create User
Update Order
Delete Payment

Query Side

Handles:

- Read operations
- Search
- Reporting

Examples:

```
Get Users
Generate Reports
Dashboard APIs
```

Real Banking Example

In banking systems:

- Transactions = write-heavy
- Reports/dashboard = read-heavy

CQRS improves:

- scalability
- performance
- maintainability

Example Without CQRS

```
Java
@Service
class UserService {
    createUser(){}
    updateUser(){}
    getUsers(){}
    getDashboard(){}
}
```

Single service handles everything.

Example With CQRS

Command Service

```
Java
@Service
class UserCommandService {

    createUser(){}

    updateUser(){}

}
```

Query Service

```
Java
@Service
class UserQueryService {

    getUsers(){}

    getDashboard(){}

}
```

CQRS Benefits

Benefit	Description
Scalability	Separate read/write scaling
Performance	Optimized queries
Maintainability	Cleaner architecture
Flexibility	Different DBs possible

Advanced CQRS Architecture

Sometimes:

- Write DB = PostgreSQL
- Read DB = Elasticsearch/Redis

Very common in large systems.

CQRS + Event Driven Architecture

Very advanced interview topic.

Flow:

```
Write DB updated
↓
Event generated
↓
Read DB synchronized
```

Used with:

- Kafka
- RabbitMQ

Difference Between CORS and CQRS

CORS	CQRS
Browser security mechanism	Architecture pattern
Controls cross-origin requests	Separates reads/writes
Frontend/backend communication	Backend design
Security concern	Scalability concern

Most Asked CQRS Interview Questions

1. What is CQRS?

Architectural pattern separating read and write operations.

2. Why use CQRS?

To improve:

- scalability
- performance
- maintainability

3. Difference between Command and Query?

Command	Query
Modifies data	Reads data

4. Can CQRS use different databases?

Yes.

Example:

- PostgreSQL for writes
- Elasticsearch for reads

5. Is CQRS always needed?

No.

Usually used in:

- Large enterprise apps
- High-scale systems
- Banking systems

6. Difference between CRUD and CQRS?

CRUD	CQRS
Single model	Separate read/write

Most Asked Real-World Questions

Since you work on enterprise applications at Aurionpro, prepare:

- CORS issues in Angular/React + Spring Boot
- Preflight requests
- API Gateway CORS handling
- CQRS with Kafka
- Event-driven microservices
- Read replicas
- Elasticsearch querying
- Eventual consistency
- Saga pattern
- Distributed transactions

These are commonly asked in modern Spring Boot microservices interviews.

Different Status codes explain all

HTTP Status Codes Complete Explanation

HTTP status codes are returned by servers to indicate:

What happened to the request

Very important for:

- REST APIs
- Spring Boot interviews
- Microservices
- Backend development

Status Code Categories

Range	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

1xx — Informational Responses

Rarely used in backend interviews.

100 Continue

Server received initial request headers.

101 Switching Protocols

Used in:

- WebSockets

2xx — Success Responses

Very important.

200 OK

Most common.

Request successful.

Example

```
Java
@GetMapping("/users")
public List<User> getUsers() {
}
```

Response:

```
200 OK
```

201 Created

Used when resource created successfully.

Example

```
Java
@PostMapping("/users")
```

New user created.

Response:

```
201 Created
```

Important Interview Point

POST APIs should ideally return:

```
201
```

not 200.

202 Accepted

Request accepted but processing not completed yet.

Used in:

- Async processing
 - Batch jobs
-

204 No Content

Request successful but no response body.

Example:

```
DELETE API
```

Example

```
Java
@DeleteMapping("/{id}")
```

Returns:

```
204 No Content
```

3xx — Redirection

301 Moved Permanently

URL permanently changed.

302 Found

Temporary redirect.

304 Not Modified

Used for caching.

Browser uses cached data.

4xx — Client Errors

Very important.

Means:

```
Client made mistake
```

400 Bad Request

Invalid request from client.

Examples:

- Missing fields
- Invalid JSON
- Validation failure

Example

```
JSON
{
  "email": "invalid"
}
```

Spring Boot Example

```
Java
@Valid
@RequestBody UserRequest request
```

Validation fails → 400.

401 Unauthorized

Authentication required.

User not logged in.

Example

```
JWT token missing
```

Important Interview Point

```
401 = Authentication issue
```

403 Forbidden

User authenticated but no permission.

Example

```
USER tries ADMIN API
```

Important Difference

401	403
Not authenticated	No permission

Very commonly asked.

404 Not Found

Requested resource not found.

Example

```
/users/999
```

User not found.

405 Method Not Allowed

Wrong HTTP method used.

Example

```
POST called on GET endpoint
```

406 Not Acceptable

Server cannot return requested format.

409 Conflict

Conflict with current resource state.

Example

```
Duplicate email registration
```

415 Unsupported Media Type

Unsupported content type.

Example

XML sent instead of JSON

422 Unprocessable Entity

Request understood but validation failed.

Used in some APIs.

429 Too Many Requests

Rate limit exceeded.

Very important in APIs.

Example

Too many login attempts

5xx — Server Errors

Means:

Backend/server issue

500 Internal Server Error

Most common server error.

Unhandled exception occurred.

Example

```
Java  
int x = 10 / 0;
```

501 Not Implemented

Feature not implemented.

502 Bad Gateway

Gateway received invalid response.

Common in:

- API Gateway
- Nginx
- Load balancers

503 Service Unavailable

Server temporarily unavailable.

Examples:

- Maintenance
- High load

504 Gateway Timeout

Gateway timeout from upstream server.

Common in microservices.

Most Important Status Codes for REST APIs

Code	Meaning
200	Success
201	Created
204	No content
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
409	Conflict
500	Internal server error

Focus heavily on these.

Real Spring Boot Example

Controller

```
Java
@PostMapping("/users")
public ResponseEntity<User> create(
    @RequestBody User user) {
```

```
User saved = service.save(user);

return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(saved);
}
```

Returns:

201 Created

ResponseEntity

Very important interview topic.

Used to customize:

- status code
- headers
- response body

Example

```
Java
return ResponseEntity
    .ok(user);
```

Common ResponseEntity Methods

Method	Status
ok()	200
created()	201
noContent()	204
badRequest()	400

Exception Handling with Status Codes

Using:

```
Java
@ControllerAdvice
```

Example

```
Java
```



```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<String> handle() {

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body("User not found");
}
```

HTTP Methods + Status Codes

Method	Common Success Code
GET	200
POST	201
PUT	200
DELETE	204

REST API Best Practices

Very important in interviews.

GET

200 OK

POST

201 Created

DELETE

204 No Content

Validation Failure

400 Bad Request

Unauthorized

401 Unauthorized

Forbidden

403 Forbidden

Common Interview Questions

1. Difference between 401 and 403?

401	403
Not authenticated	No permission

2. Difference between 200 and 201?

200	201
Success	Resource created

3. Why use 204?

For successful operations with no response body.

4. What is 500 error?

Unhandled server exception.

5. What is 502 vs 504?

502	504
Invalid upstream response	Upstream timeout

6. What is 429?

Too many requests/rate limiting.

7. What status code for validation failure?

Usually:

8. What is ResponseEntity?

Spring class for custom HTTP responses.

Most Asked Real Project Questions

Since you work on enterprise backend systems at Aurionpro, expect:

- Proper REST status code usage
- Global exception handling
- API standardization
- Error response design
- Validation handling
- Gateway timeout issues
- Retry mechanisms
- Circuit breaker failures
- Rate limiting
- Idempotency

These are extremely common in Spring Boot interviews for 2-5 years experience.

What is an API?
 Why we use API's
 Different types of API's with examples why we need drawbacks and all
 Which API to use where
 Explain all with examples and interview questions

What is an API?

Application Programming Interface API is:

A way for two software systems to communicate with each other.

Simple Real-Life Example

Imagine:

- Restaurant waiter = API
- Customer = Client

- Kitchen = Server

Flow:

```
Customer gives order
↓
Waiter takes request
↓
Kitchen prepares food
↓
Waiter returns response
```

API acts like waiter between systems.

In Software

Frontend:

```
React / Angular / Mobile App
```

calls backend API:

```
/users
/payments
/orders
```

Backend returns data.

Example API

Request

```
http
GET /users/1
```

Response

```
JSON
{
  "id": 1,
  "name": "Lawrence"
}
```

Why APIs Are Used

Very important interview question.

1. Communication Between Systems

Frontend ↔ Backend

2. Reusability

Same API used by:

- Web app
 - Mobile app
 - Third-party systems
-

3. Scalability

Microservices communicate using APIs.

4. Integration

Used for:

- Payment gateways
 - Banking systems
 - Google Maps
 - SMS providers
-

5. Security

Backend logic hidden from frontend.

API Architecture Flow

```

Client
↓
API Request
↓
Backend Service
↓
Database
↓
API Response
  
```

Types of APIs

Very important interview topic.

1. REST API

Most commonly used today.

Representational State Transfer

Uses:

- HTTP methods
- JSON
- Stateless communication

Example

```
http
GET /users
POST /users
PUT /users/1
DELETE /users/1
```

Features

Feature	REST
Protocol	HTTP
Data Format	JSON
Stateless	Yes
Lightweight	Yes

Why REST Popular

- Simple
- Fast
- Scalable
- Easy frontend integration

Drawbacks

- Over-fetching possible
- Multiple API calls needed sometimes

Best Used For

- Web applications
- Mobile apps
- Microservices

Most used in Spring Boot.

2. SOAP API

Simple Object Access Protocol

Old enterprise protocol.

Uses:

- XML
- Strict standards

Example SOAP Request

```
XML
<soap:Envelope>
  <soap:Body>
    <getUser>
    </getUser>
  </soap:Body>
</soap:Envelope>
```

Features

Feature	SOAP
Data Format	XML
Security	Strong
Standardized	Yes
Heavyweight	Yes

Advantages

- High security
- ACID transactions
- Enterprise standards

Drawbacks

- Slow
- Complex XML
- Heavy payload

Best Used For

- Banking systems
- Legacy enterprise apps
- Payment systems

Still used in banks heavily.

REST vs SOAP

Very commonly asked.

REST	SOAP
Lightweight	Heavyweight
JSON	XML
Faster	Slower
Simple	Complex
Stateless	Can be stateful
Modern apps	Legacy enterprise

3. GraphQL API

GraphQL created by Meta Platforms.

Allows client to request exactly needed data.

Example

```
GraphQL
{
  user(id:1){
    name
    email
  }
}
```

Benefits

- No over-fetching

- Single endpoint
- Flexible queries

Drawbacks

- Complex backend
- Caching difficult

Best Used For

- Complex frontend apps
- Large UI systems
- Mobile optimization

REST vs GraphQL

REST	GraphQL
Multiple endpoints	Single endpoint
Fixed response	Flexible response
Simpler	More complex

4. gRPC

gRPC developed by Google.

Uses:

- Protocol Buffers
- HTTP/2

Very fast.

Features

Feature	gRPC
Speed	Very fast
Data Format	Binary
Streaming	Supported

Advantages

- High performance
- Small payload
- Efficient communication

Drawbacks

- Hard browser support
- Complex debugging

Best Used For

- Microservices communication
- Internal backend systems

5. WebSocket API

Provides:

Real-time communication

Examples

- Chat apps
- Live trading
- Gaming
- Notifications

Difference from REST

REST	WebSocket
Request-response	Full duplex
Stateless	Persistent connection

6. Open APIs / Public APIs

Available publicly.

Examples:

- Google Maps API
- Weather API
- Payment Gateway APIs

7. Internal APIs

Used inside organization.

Example:

Microservice communication

HTTP Methods in REST APIs

Very important.

Method	Purpose
GET	Fetch
POST	Create
PUT	Update
DELETE	Delete
PATCH	Partial update

Example

http
GET /users

Fetch users.

API Status Codes

Code	Meaning
200	Success
201	Created
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
500	Server error

API Security

Very important interview topic.

Common Security Methods

Method	Usage
JWT	Modern APIs
OAuth2	Google login
API Keys	Third-party APIs
Basic Auth	Simple systems

API Versioning

Very commonly asked.

Example

```
/api/v1/users
/api/v2/users
```

Why Needed?

Avoid breaking old clients.

Idempotency

Very important REST concept.

Idempotent Methods

Calling multiple times gives same result.

Method	Idempotent
GET	Yes
PUT	Yes
DELETE	Yes
POST	No

API Gateway

Very important in microservices.

Acts as:

Single entry point

Handles:

- Authentication
- Routing
- Rate limiting
- Logging

Examples:

- Kong
- Nginx
- Spring Cloud Gateway

API Documentation

Tools:

- Swagger/OpenAPI

Very important in Spring Boot.

Example Swagger Dependency

```
XML
springdoc-openapi-ui
```

Most Important Interview Questions

Basic Questions

1. What is API?

API is a communication interface between software systems.

2. Why APIs are used?

- Integration
- Reusability
- Scalability

3. Difference between REST and SOAP?

| REST | SOAP |

4. What is stateless API?

Server stores no client session.

5. Difference between PUT and PATCH?

PUT	PATCH
Full update	Partial update

Intermediate Questions

6. What is idempotency?

Repeated requests produce same result.

7. Why API versioning needed?

To support backward compatibility.

8. What is API Gateway?

Single entry point for microservices.

9. Difference between REST and GraphQL?

| REST | GraphQL |

10. What is gRPC?

High-performance RPC framework.

Advanced Questions

11. How REST APIs work internally?

HTTP request-response architecture.

12. What is HATEOAS?

REST maturity concept with hyperlinks.

13. What is rate limiting?

Restricting excessive API requests.

14. What is throttling?

Controlling request frequency.

15. What is circuit breaker?

Prevents cascading failures.

Which API to Use Where?

Very important practical question.

Use REST API

When:

- Standard web/mobile apps
- Public APIs
- CRUD operations

Most common choice.

Use SOAP

When:

- Enterprise security needed
 - Legacy banking systems
 - Strict contracts
-

Use GraphQL

When:

- Frontend needs flexible data
 - Complex UI applications
-

Use gRPC

When:

- High-speed internal communication
 - Microservices backend communication
-

Use WebSocket

When:

- Real-time apps needed
-

Real Industry Usage

Since you're working in enterprise Java at Aurionpro, you'll likely encounter:

- REST APIs
- SOAP integrations

- JWT authentication
- API Gateway
- Swagger/OpenAPI
- Kafka event APIs
- Microservices communication
- Rate limiting
- Webhooks
- API versioning
- OAuth2 security

These are heavily asked in Spring Boot interviews for 2-5 years experience.

how does graphql and websocket apis work

How GraphQL Works

GraphQL is an API query language developed by Meta Platforms.

It allows clients to:

Request exactly the data they need

Problem with REST APIs

Suppose frontend needs:

- name
- email

But REST API returns:

```
JSON
{
  "id": 1,
  "name": "Lawrence",
  "email": "abc@gmail.com",
  "address": "Mumbai",
  "phone": "9999999999"
}
```

Extra unused data returned.

Called:

Over-fetching

Another Problem

Frontend may need:

- User details
- Orders
- Payments

REST may require:

Multiple API calls

Called:

Under-fetching

Solution — GraphQL

Client asks only required fields.

Example Query

```
GraphQL
{
  user(id:1){
    name
    email
  }
}
```

Response

```
JSON
{
  "data":{
    "user":{
      "name":"Lawrence",
      "email":"abc@gmail.com"
    }
  }
}
```

Only needed data returned.

GraphQL Architecture

```
Frontend
↓
GraphQL Query
↓
GraphQL Server
↓
Resolver
```

↓
Database/API

Important GraphQL Concepts

1. Schema

Defines:

- Available queries
- Data types
- Relationships

Example

```
GraphQL
type User {
  id: ID
  name: String
  email: String
}
```

2. Query

Used for:

READ operations

Example

```
GraphQL
{
  users {
    name
  }
}
```

3. Mutation

Used for:

WRITE operations

Like:

- insert
- update

- delete

Example

```
GraphQL
mutation {
  createUser(name: "Lawrence"){
    id
  }
}
```

4. Resolver

Very important interview topic.

Resolver:

Executes GraphQL query logic

Equivalent to controller/service in REST.

Example Flow

```
Query
↓
Resolver
↓
Service
↓
Database
```

5. Subscription

Used for:

Real-time updates

Uses WebSockets internally.

Example:

- Live chat
- Stock prices

GraphQL Request Flow

Step 1

Frontend sends:

```
GraphQL
{
  user(id:1){
    name
  }
}
```

Step 2

GraphQL parser validates query.

Step 3

Resolver executes logic.

Step 4

Data fetched from DB.

Step 5

Only requested fields returned.

GraphQL vs REST

Very commonly asked.

REST	GraphQL
Multiple endpoints	Single endpoint
Fixed response	Flexible response
Over-fetching possible	Exact data
Easier caching	Complex caching

Advantages of GraphQL

Benefit	Description
Flexible	Client chooses data
Efficient	Less data transfer
Single endpoint	Cleaner architecture
Faster frontend development	Reduced API changes

Drawbacks of GraphQL

Problem	Description
Complex backend	Harder implementation
Caching difficult	Dynamic responses
Query optimization needed	Expensive nested queries

When to Use GraphQL

Best for:

- Complex frontend apps
- Mobile apps
- Large UI systems
- Multiple frontend teams

When NOT to Use GraphQL

Avoid when:

- Simple CRUD app
- Small APIs
- Simple backend

REST is easier there.

Spring Boot GraphQL Example

Dependency:

```
XML
spring-boot-starter-graphql
```

Resolver Example

```
Java
@Controller
public class UserResolver {

    @QueryMapping
    public User getUser(@Argument Long id) {
```

```

    }
    return service.findById(id);
}

```

Most Asked GraphQL Interview Questions

1. What problem does GraphQL solve?

- Over-fetching
- Under-fetching

2. Difference between Query and Mutation?

Query	Mutation
Read	Write

3. What is Resolver?

Executes query logic.

4. Why GraphQL has single endpoint?

Because query structure determines response.

5. Difference between REST and GraphQL?

Very commonly asked.

How WebSocket Works

WebSocket provides:

Real-time bidirectional communication

between:

- client

- server

Problem with REST

REST works like:

```
Request
↓
Response
↓
Connection closed
```

Not suitable for real-time apps.

Real-Time Examples

- Chat apps
- Live cricket score
- Trading systems
- Notifications
- Gaming

WebSocket Solution

Persistent connection remains open.

WebSocket Flow

```
Client
↑
Persistent Connection
↑
Server
```

Both can send messages anytime.

Difference Between HTTP and WebSocket

HTTP	WebSocket
Request-response	Full duplex
Stateless	Persistent
Repeated connections	Single connection
Slower real-time	Fast real-time

WebSocket Handshake

Very important interview topic.

Step 1

Client sends:

```
http
Upgrade: websocket
```

Step 2

Server accepts upgrade.

Step 3

HTTP connection upgraded to:

```
WebSocket connection
```

Persistent connection established.

Full Duplex Communication

Means:

```
Both client and server can send data anytime
```

Example

Chat app:

- Client sends message
- Server instantly broadcasts

No repeated API calls.

Spring Boot WebSocket Example

Dependency:

```
XML
```

Configuration

```
Java
@EnableWebSocketMessageBroker
@Configuration
public class WebSocketConfig {
}
```

Endpoint

```
Java
registry.addEndpoint("/chat");
```

Message Mapping

```
Java
@MessageMapping("/message")
@SendTo("/topic/messages")
public String send(String msg) {
    return msg;
}
```

STOMP Protocol

Very commonly asked.

Used over WebSocket.

Provides:

- publish/subscribe model

Example

```
Client subscribes:
/topic/messages
```

Server pushes messages automatically.

WebSocket Advantages

Benefit	Description
Real-time	Instant updates
Efficient	Less overhead
Persistent	Single connection

WebSocket Drawbacks

Problem	Description
Hard scaling	Stateful connections
More server memory	Open connections
Complex handling	Reconnection issues

WebSocket vs REST

REST	WebSocket
CRUD APIs	Real-time systems
Stateless	Persistent
Simple	More complex

WebSocket vs SSE

WebSocket	SSE
Two-way	One-way
Full duplex	Server → Client only

Most Asked WebSocket Interview Questions

1. What is WebSocket?

Protocol for real-time bidirectional communication.

2. Difference between HTTP and WebSocket?

| HTTP | WebSocket |

3. What is handshake in WebSocket?

HTTP connection upgrade process.

4. What is STOMP?

Messaging protocol over WebSocket.

5. Why use WebSocket?

Real-time communication.

6. Can REST replace WebSocket?

No.

REST unsuitable for real-time communication.

When to Use Which?

Use REST

- CRUD APIs
 - Standard backend systems
-

Use GraphQL

- Complex frontend requirements
 - Flexible data fetching
-

Use WebSocket

- Real-time systems
 - Live updates
 - Chat/trading
-

Real Industry Usage

Since you work on enterprise Java applications at Aurionpro, expect questions on:

- REST vs GraphQL
- WebSocket vs Kafka

- STOMP
- Real-time dashboards
- Event-driven systems
- Push notifications
- Live transaction monitoring
- API optimization
- Subscription models
- Streaming systems

These are increasingly common in modern Spring Boot interviews.